Linear Time Minimum Area All-flush Triangles Circumscribing a Convex Polygon

Kai Jin[1] **and Zhiyi Huang**[2]

1 **Department of Computer Science, University of Hong Kong**
cscjjk@gmail.com
2 **Department of Computer Science, University of Hong Kong**
hzhiyi.tcs@gmail.com

Abstract

We study the problem of computing the minimum area triangle that circumscribes a given n-sided convex polygon touching edge-to-edge. In other words, we compute the minimum area triangle that is the intersection of 3 half-planes out of n half-planes defined by a given convex polygon. Building on the Rotate-and-Kill technique [11], we propose an algorithm that solves the problem in $O(n)$ time, improving the best-known $O(n \log n)$ time algorithms given in [2, 3, 18]. Our algorithm computes all the locally minimal area circumscribing triangles touching edge-to-edge.

1998 ACM Subject Classification I.3.5 Computational Geometry and Object Modeling

Keywords and phrases Geometric optimization, Polygon inclusion problem, Convex polygon, Rotate-and-Kill technique, Algorithm

Digital Object Identifier 10.4230/LIPIcs..2016.23

1 Introduction

Given a convex polygon P with n edges, the *minimum all-flush k-gon problem* asks for the minimum (with respect to area throughout this paper) k-gon whose edges are all flushed with edges of P (i.e. each edge must contain an edge of P as a subregion). In other words, it asks for the minimum k-gon that circumscribes P touching P edge-to-edge.

This problem was proposed by Aggarwal et al. [3]. They solved it in $O(n\sqrt{k \log n} + n \log n)$ time using their technique for computing the *minimum weight k-link path*. For $k = \Omega(\log n)$, this improves over an $O(kn + n \log n)$ time algorithm based on the *matrix-search technique* [2] (however, $O(kn)$ is better when k is regarded as a constant as discussed below), which improves over an $O(kn \log n + n \log^2 n)$ time algorithm implicitly given in [5]. All these previous work follow the same approach: first, choose an arbitrary edge e of P and find the minimum all-flush k-gon Q flushed by e, which reduces to solving an instance of the minimum weight k-link path problem; second, compute the minimum all-flush k-gon from Q. The term $n\sqrt{k \log n}$ comes from the first step and $n \log n$ comes from the second. Schieber [18] slightly improves the first term by optimizing the underlying technique for computing the minimum weight k-link path. Yet the second term $n \log n$ is unchanged. To the best of our knowledge, no one has improved this part even for the simplest case of $k = 3$.

However, for $k = 3$, we can solve the dual problem, i.e. computing the maximum area triangle (MAT) inside a convex polygon, in $O(n)$ time [11, 7]. So, it is interesting to know whether we can also compute in linear time the minimum all-flush triangle (MFT).

We settle this question affirmatively in this paper by improving the aforementioned second term to n for $k = 3$. In our algorithm, after computing Q, we first compute another triangle Q' from Q and then compute the MFT from Q', both in linear time. (However, note that

computing Q and Q' will be put together in this paper and referred to as the initial step of our algorithm. The main difficulty lies in computing the MFT from Q'.)

The MFT problem is as fundamental as the *minimum enclosing triangle* problem studied in history [14, 16, 7] and may find similar applications in more realistic problems. By computing the MFT, we obtain a simple container of P to accelerate the polygon collision detection. Moreover, it can be applied in finding a good packing of P into the plane. In the packing problem, we want to pack non-overlap copies of P in the plane, so that the ratio between the uncovered area and the area covered by the copies is as small as possible.

Literature of the "dual" problem. For the MAT problem, there is a well-known linear time algorithm given by Dobkin and Snyder [9], which was found **incorrect** by Keikha et al. [13] recently. Nonetheless, there is a correct but more involved linear solution given by Chandran and Mount [7] based on the *rotating-caliper technique* [19]. Jin [11] recently reported another linear time algorithm, which is much simpler than the one in [7]. Another algorithm was reported by Kallus [12]. Although, the MFT and MAT problems are often viewed as dual (from the combinatorial perspective) [3, 18], to our knowledge, there is no reduction from an instance of the MFT problem to an instance of the MAT problem that allows us to translate an algorithm of the latter to the former. See a discussion in appendix B.

Rotate-and-Kill technique. Jin [11] introduced a so-called *Rotate-and-Kill technique* for solving the polygon inclusion problem, which will be applied in this paper for finding the MFT. So, let us briefly review how this technique is applied on the MAT problem.

Consider a naïve algorithm for finding the MAT: enumerate a vertex pair (V, V') of P and computes the vertex V^* so that the area of $\triangle VV'V^*$ is maximum. It suffers from enumerating too many pairs of (V, V'). In fact, only a few of these pairs are effective as implied by the following iterative process called Rotate-and-Kill. Let $V + 1$ denote the clockwise next vertex of V. Jin [11] designed a constant time subroutine $\mathsf{Kill}(V, V')$, called *killing criterion*, which returns either V or V', so that V is returned only if (a) no pair in $\{(V, V' + 1), (V, V' + 2), \ldots\}$ forms an edge of an MAT and V' is returned only if (b) no pair in $\{(V + 1, V'), (V + 2, V'), \ldots\}$ forms an edge of an MAT. Now, assume a pair (V, V') is given in the current iteration. We kill V if $\mathsf{Kill}(V, V') = V$ and otherwise kill V', and then move on to the next iteration $(V + 1, V')$ or $(V, V' + 1)$. In this way, only $O(n)$ pairs of (V, V') are enumerated and the algorithm is thus improved to $O(n)$ time.

In addition to the MAT, [11] also computes the minimum enclosing triangle optimally by this new technique. Naturally, [11] guesses that their technique is powerful for solving other related problems. A precondition for applying the Rotate-and-Kill technique is that at least one of (a) and (b) holds at any iteration. This is indeed truth for many polygon inclusion problems since the locally optimal solutions in such problems usually admit an interleaving property (see [5] or Definition 5 below) implying that (a) and (b) cannot fail simultaneously.

When the above precondition is satisfied for a given problem, the biggest challenge in applying the technique is that we need an efficient killing criterion specialized to the problem. Usually, a criterion that runs in $O(\mathsf{poly}(\log n))$ or even $O(\log n)$ time is easy to find. Yet we wish to have an (amortized) $O(1)$ time criterion as shown in [11]. For the MFT problem, although we can borrow the framework in [11], we must settle this main challenge by developing new ideas. In fact, our criterion is more tricky than the one in [11].

Other related work. Searching for extremal shapes with special properties enclosing or enclosed by a given polygon were initiated in [9, 5, 8], and have since been studied extensively. Chandran and Mount's algorithm [7] is an extension of O'Rourke et al.'s linear time algorithm [16] for computing the minimum triangle enclosing P. The latter is an improvement over

an algorithm of Klee and Laskowski [14]. The minimum perimeter enclosing triangle can be solved in $O(n)$ time [4]. The maximum perimeter enclosed triangle can be solved in $O(n \log n)$ time [5]. [5, 2, 3, 18, 8, 1, 15] studied extremal area / perimeter k-gon inside or outside a convex polygon. In particular, the maximum k-gon can be computed in $O(n \log n)$ time when k is a constant [2, 3, 18] and it remains open whether this can be optimized to linear time (at least for $k = 4$). [21, 20] studied the extremal polytope problems in three dimensional space. Brass and Na [6] solves another related problem: Given n half-planes (in arbitrary position), find the maximum bounded intersection of k half-planes out of them. We refer the readers to the introduction of [10] and [11] for more related work.

Key motivation. The well-known rotating-caliper technique is powerful in solving a lot of polygon enclosing problems, but not easy to apply in most polygonal inclusion problems. To our knowledge, there was no generic technique for solving the polygon inclusion problem as claimed in [17] before the Rotate-and-Kill technique (noticing that [9] is wrong). Thus, for attacking the inclusion problems, there is a necessity to further develop the unmature Rotate-and-Kill technique, especially by finding more of its applications. This motivates us to study the MFT problem in this paper (even though it is actually a polygon enclosing problem). Nonetheless, we believe that our result brings some new understanding of the technique that might be helpful for improving other related problems.

1.1 Preliminaries

Let $v_1, \ldots, v_n$ be a clockwise enumeration of the vertices of the given convex polygon P. For each i, denote by e_i the directed line segment $\overrightarrow{v_i v_{i+1}}$. We call $e_1, \ldots, e_n$ the n edges of P. Assume that no three vertices of P lie in the same line and moreover, all edges of P are **pairwise-nonparallel**. Let ℓ_i denote the extended line of e_i, and p_i denote the half-plane delimited by ℓ_i and containing P, and p_i^C denote the complementary half-plane of p_i.

When three distinct edges e_i, e_j, e_k lie in clockwise order, the region bounded by $\mathsf{p}_i, \mathsf{p}_j, \mathsf{p}_k$ is denoted by $\triangle e_i e_j e_k$ and is called an *all-flush triangle*. Throughout, whenever we write $\triangle e_i e_j e_k$, we assume that e_i, e_j, e_k *are distinct and lie in clockwise order*.

Denote the area of $\triangle e_i e_j e_k$ by $\mathsf{Area}(\triangle e_i e_j e_k)$. This area may be unbounded. We can use the following observation to determine the finiteness of $\mathsf{Area}(\triangle e_i e_j e_k)$.

▶ **Definition 1** (Chasing relation)**.** Edge e_i is *chasing* another edge e_j, denoted by $e_i \prec e_j$, if the intersection of ℓ_i, ℓ_j lies between e_i, e_j clockwise.

▶ **Observation 2.** $\mathsf{Area}(\triangle e_i e_j e_k)$ *is finite if and only if:* $e_i \prec e_j, e_j \prec e_k$ *and* $e_k \prec e_i$.

▶ **Observation 3.** *There exists a tuple* (e_i, e_j, e_k) *such that* $e_i \prec e_j, e_j \prec e_k$ *and* $e_k \prec e_i$.

Proof. Choose e_i arbitrarily. Choose j so that $e_i \prec e_j$ but $e_{j+1} \prec e_i$. Let $k = j + 1$. ◀

For the all-flush triangles with finite areas, we can define the notion of *3-stable*. (Note that finiteness is a prerequisite of being 3-stable because otherwise subsequent lemmas, e.g. Lemma 6, would fail or be too complicated to state; see discussions in Appendix B.)

▶ **Definition 4.** Consider any all-flush triangle $\triangle e_i e_j e_k$ with a finite area. Edge e_i is *stable* if no all-flush triangle $\triangle e_{i'} e_j e_k$ is smaller than $\triangle e_i e_j e_k$; edge e_j is *stable* if no all-flush triangle $\triangle e_i e_{j'} e_k$ is smaller than $\triangle e_i e_j e_k$; and edge e_k is *stable* if no all-flush triangle $\triangle e_i e_j e_{k'}$ is smaller than $\triangle e_i e_j e_k$. Moreover, triangle $\triangle e_i e_j e_k$ is *3-stable* if e_i, e_j, e_k are all stable.

Combining Observation 2 and 3, there exist all-flush triangles with finite areas. Moreover, by Definition 4, if a finite all-flush triangle is not 3-stable, we could find a smaller such triangle. Therefore, to find the minimum area all-flush triangle, it suffices if we first compute all the 3-stable triangles and then select the minimum among them.

Below we introduce the notion of *interleaving* and an important property of 3-stable triangles, whose corollary shows that there are not too many such triangles.

▶ **Definition 5.** Two flushed triangles $\triangle e_r e_s e_t$ and $\triangle e_i e_j e_k$ are *interleaving* if there is a list of edges $e_{a_1}, \ldots, e_{a_6}$ which lie in clockwise order (in a non-strict manner; so neighbors may be identical), in which $\{e_{a_1}, e_{a_3}, e_{a_5}\}$ equals $\{e_r, e_s, e_t\}$ and $\{e_{a_2}, e_{a_4}, e_{a_6}\}$ equals $\{e_i, e_j, e_k\}$.

▶ **Lemma 6.** *Any two 3-stable triangles are interleaving.*

▶ **Corollary 7.** *There are $O(n)$ 3-stable triangles.*

Easy proofs of Lemma 6 and Corollary 7 are deferred to Appendix A.

1.2 Overview of our approach

Initial step. We first compute one 3-stable triangle by a somewhat trivial algorithm. Denote the resulting 3-stable triangle by $\triangle e_r e_s e_t$. Let $J = \{e_s, \ldots, e_t\}$ and $K = \{e_t, \ldots, e_r\}$, where $\{e_x, \ldots, e_y\}$ denotes the set of edges between e_x to e_y clockwise including e_x and e_y.

The naïve approach. For each 3-stable triangle $\triangle e_i e_j e_k$, since it must interleave $\triangle e_r e_s e_t$ (by Lemma 6), we can assume without loss of generality that $e_j \in J$ and $e_k \in K$. Therefore, the following algorithm computes all the 3-stable triangles: Enumerate $(e_b, e_c) \in J \times K$ and for each such edge pair, compute the 3-stable triangle(s) $\triangle e_i e_j e_k$ with $(e_j, e_k) = (e_b, e_c)$. However, this algorithm costs $\Omega(|J \times K|)$ time, which is $\Omega(n^2)$ in worst (and most) cases.

We say (e_b, e_c) is *dead* if there does not exist an edge e_a such that $\triangle e_a e_b e_c$ is 3-stable. Clearly, it is unnecessary to enumerate a dead pair in the above algorithm. Further, there are only $O(n)$ pairs that are not dead according to Corollary 7. Therefore, the above algorithm could be improved if those pairs that are not dead can be found efficiently.

Rotate-and-Kill. Initially, set $(b, c) = (s, t)$, i.e. set e_b, e_c to be the first edges in J, K respectively. Iteratively, choose one of the following operations:

Kill b (i.e. $b \leftarrow b + 1$); or kill c (i.e. $c \leftarrow c + 1$). Obey the following rules.

b is killed only if (1) the pairs in $\{(e_b, e_{c+1}), (e_b, e_{c+2}), ..., (e_b, e_r)\}$ are all dead, and
c is killed only if (2) the pairs in $\{(e_{b+1}, e_c), (e_{b+2}, e_c), ...(e_t, e_c)\}$ are all dead.

The termination condition is $(b, c) = (t, r)$, i.e. e_b, e_c are the last edges in J, K respectively.

Suppose both rules are obeyed, the iteration would eventually reach the state $(b, c) = (t, r)$ and at that moment *all the pairs that are not dead would have been enumerated.* To see this more clearly, observe that (e_t, e_r) is not dead (because $\triangle e_s e_t e_r$ is 3-stable), and observe that by induction, at each iteration of (b, c), an pair $(e_{b'}, e_{c'})$ that is not dead either has been enumerated already or satisfies that $e_{b'} \in \{e_b, \ldots, e_t\}$ and $e_{c'} \in \{e_c, \ldots, e_r\}$.

The above Rotate-and-Kill process shall be finalized with a function $\mathsf{Kill}(b, c)$, called *killing criterion*, which guides us to kill b or c. It returns b only if (1) holds and c only if (2) holds. Above all, notice that such a criterion does exist. This is because (1) or (2) holds at each iteration. Suppose neither (1) nor (2) in some iteration and without loss of generality that (e_b, e_{c+g}) and (e_{b+h}, e_c) are not dead. This suggests two 3-stable triangles $\triangle e_{a_1} e_b e_{c+g}$ and $\triangle e_{a_2} e_{b+h} e_c$, which definitely cannot be interleaving and thus contradicts Lemma 6.

The criterion is the kernel of the algorithm; designing it is the crucial part of the paper.

Logarithmic killing criterion. Two criterions obey the rules: Return b when (1) holds and c otherwise. Or, return c when (2) holds or b otherwise. Yet they are not computationally efficient. Computing (1) (or (2)) costs $O(n)$ time by trivial methods, or $O(\log n)$ time by binary searches (see Appendix C). These can only lead to $O(n^2)$ or $O(n \log n)$ time solutions.

Amortized constant time killing criterion. We design an amortized $O(1)$ time killing criterion in Section 3. Briefly, given (b, c), we compute a specific directed line $L = L_{b,c}$ (in $O(1)$ time) and compare it with P. Then, return b or c depending on whether P lies on the right of L. We make sure that the slope of L monotonously increase throughout the entire algorithm, thus it only costs amortized $O(1)$ time to compare the convex polygon P with L.

Compute the 3-stable triangle(s). It remains to specify how we compute the 3-stable triangle $\triangle e_i e_j e_k$ with $(j, k) = (b, c)$ in (amortized) constant time. We first compute $\mathsf{a}_{b,c} = a$ so that $\mathsf{Area}(\triangle e_a e_b e_c)$ is minimum (see Definition 8 below for a rigorous definition of $\mathsf{a}_{b,c}$), and then check whether $\triangle e_a e_b e_c$ is 3-stable and report it if so. We apply two basic lemmas here. The unimodality of $\mathsf{Area}(\triangle e_a e_b e_c)$ for fixed b, c (Lemma 9) states that if e_a is enumerated clockwise along the interval of edges for which $\triangle e_a e_b e_c$ is all-flush and $\mathsf{Area}(\triangle e_a e_b e_c)$ is finite, this area would first decrease and then increase. The bi-monotonicity of $\mathsf{a}_{b,c}$ (Lemma 10) states that if e_b or e_c moves clockwise along the boundary of P, so does $\mathsf{a}_{b,c}$. Because e_b, e_c move clockwise during the Rotate-and-Kill process, $\mathsf{a}_{b,c}$ moves clockwise by the bi-monotonicity and thus can be computed using the unimodality in amortized $O(1)$ time. Checking 3-stability (not necessary) reduces to checking whether e_a, e_b, e_c are stable in this triangle, which only takes $O(1)$ time also by the unimodality.

A pseudo code of our main algorithm is given in Appendix B.

2 Compute one 3-stable triangle

To present our algorithm, we first give two basic lemmas mentioned in the last paragraph of Subsection 1.2. Their easy proofs are put into Appendix A due to space limit.

For each i, let D_i denote the vertex with the furthest distance to ℓ_i. Given points X, X' on the boundary of P, we denote by $(X \circlearrowright X')$ the boundary portion of P that starts from X and clockwise to X' which does not contain endpoints X, X'.

▶ **Definition 8.** Consider any edge pair (e_b, e_c) such that $e_b \prec e_c$. Notice that "$e_a \prec e_b$ and $e_c \prec e_a$" is equivalent to "$e_a \in (\mathsf{D}_b \circlearrowright \mathsf{D}_c)$". We define $\mathsf{a}_{b,c}$ to be the **smallest** (i.e. clockwise first) a such that $\mathsf{Area}(\triangle e_a e_b e_c) = \min(\mathsf{Area}(\triangle e_a e_b e_c) \mid e_a \in \{e_{c+1}, \ldots, e_{b-1}\})$. For the special case where $\mathsf{D}_b = \mathsf{D}_c$, $\mathsf{Area}(\triangle e_a e_b e_c)$ are infinite for all $e_a \in \{e_{c+1}, \ldots, e_{b-1}\}$ by Observation 2 and we define $\mathsf{a}_{b,c}$ to be the previous edge of D_b. See Figure 1 below.

Sometimes we adopt the convention to abbreviate e_i as i. Hence $\mathsf{a}_{b,c}$ denotes $e_{\mathsf{a}_{b,c}}$.

▶ **Lemma 9** (Unimodality of $\mathsf{Area}(\triangle e_a e_b e_c)$ for fixed b, c)**.** *Given b, c so that $e_b \prec e_c$ and $\mathsf{D}_b \neq \mathsf{D}_c$, function $\mathsf{Area}(\triangle e_a e_b e_c)$ is unimodal for $e_a \in (\mathsf{D}_b \circlearrowright \mathsf{D}_c)$. Specifically, this function*

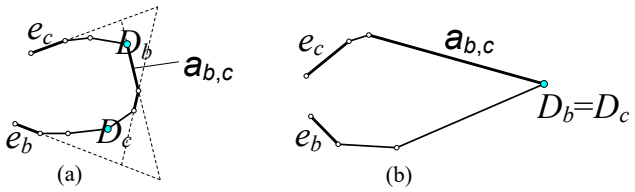

Figure 1 Illustration of the definition of $\mathsf{a}_{b,c}$.

strictly *decreases when e_a is enumerated clockwise from the next edge of D_b to $\mathsf{a}_{b,c}$; and for $e_a = \mathsf{a}_{b,c}$, we have $\mathsf{Area}(\triangle e_{a+1}e_be_c) \geq \mathsf{Area}(\triangle e_ae_be_c)$; and it* strictly *increases when e_a is enumerated clockwise from the* next edge of $\mathsf{a}_{b,c}$ *to the previous edge of D_c.*

▶ **Lemma 10** (Bi-monotonicity of $\mathsf{a}_{b,c}$)**.** *Let E denote $\{e_{c+1}, \ldots, e_{b-1}\}$ in the following claims.*

1. *Assume e_b is chasing e_c, e_{c+1}, so $\mathsf{a}_{b,c}, \mathsf{a}_{b,c+1}$ are defined. Notice that these two edges lie in E according to Definition 8. We claim that $\mathsf{a}_{b,c}, \mathsf{a}_{b,c+1}$ lie in clockwise order in E.*
2. *Assume e_b, e_{b+1} are chasing e_c, so $\mathsf{a}_{b,c}, \mathsf{a}_{b+1,c}$ are defined. Notice that these two edges lie in E according to Definition 8. We claim that $\mathsf{a}_{b,c}, \mathsf{a}_{b+1,c}$ lie in clockwise order in E.*

Here, "lie in clockwise order" is in a non-strict manner; which means equal is allowed.

To find a 3-stable triangle, our first goal is to find a triangle with two stable edges. We find it as follows. Assign $r = 1$, enumerate an edge e_s clockwise and compute $t_s = \mathsf{a}_{r,s}$ for each s, and then select s so that $\mathsf{Area}(\triangle e_re_se_{t_s})$ is minimum. In other words, we compute s, t so that $\triangle e_re_se_t$ is the smallest all-flush triangle with $r = 1$. Using the bi-monotonicity of $\{\mathsf{a}_{r,s}\}$ (Lemma 10) with the unimodality of $\mathsf{Area}(\triangle e_re_se_t)$ for fixed r, s (Lemma 9), the computation of t_s only costs amortized $O(1)$ time, hence the entire running time is $O(n)$.

We claim that $\triangle e_re_se_t$ has a finite area and moreover, e_s, e_t are stable in $\triangle e_re_se_t$. By Observation 2 and the proof of Observation 3, for any given edge e_r, there exist e_j, e_k so that $\triangle e_re_je_k$ has a finite area. This easily implies the finiteness of $\triangle e_re_se_t$. If e_s (or e_t) is not stable in $\triangle e_re_se_t$, we could get a smaller triangle $\triangle e_re_{s'}e_t$ (or $\triangle e_re_se_{t'}$) with $r = 1$, contradicting the fact that $\triangle e_re_se_t$ is the smallest all-flush triangle with $r = 1$.

Now, e_s and e_t are stable in $\triangle e_re_se_t$. If e_r is also stable (which can be determined in $O(1)$ time by Lemma 9), we have found a 3-stable triangle. What if e_r is not stable? By Lemma 9, this means either $\mathsf{Area}(\triangle e_{r+1}e_se_t) < \mathsf{Area}(\triangle e_re_se_t)$ or $\mathsf{Area}(\triangle e_{r-1}e_se_t) < \mathsf{Area}(\triangle e_re_se_t)$. **Assume the former occurs** and our subroutine for this case is given in Algorithm 1. The latter can be handled by a symmetric subroutine as shown in Algorithm 3.

```
1 while r + 1 ≠ s and Area(△e_{r+1}e_se_t) < Area(△e_re_se_t) do
2 |   r ← r + 1;
3 |   repeat
4 |   |   while s + 1 ≠ t and Area(△e_re_{s+1}e_t) < Area(△e_re_se_t) do s ← s + 1;
5 |   |   while t + 1 ≠ r and Area(△e_re_se_{t+1}) < Area(△e_re_se_t) do t ← t + 1;
6 |   until none of the above two conditions hold;
7 end
```

Algorithm 1: Find a 3-stable triangle for the case $\mathsf{Area}(\triangle e_{r+1}e_se_t) < \mathsf{Area}(\triangle e_re_se_t)$.

To analysis Algorithm 1, we introduce two notions: back-stable and forw-stable. Consider any all-flush triangle $\triangle e_ie_je_k$ with a finite area. Edge e_i is *back-stable* if $\mathsf{Area}(\triangle e_ie_je_k) \leq \mathsf{Area}(\triangle e_{i-1}e_je_k)$ (or $i - 1 = k$). Edge e_i is *forw-stable* if $\mathsf{Area}(\triangle e_ie_je_k) \leq \mathsf{Area}(\triangle e_{i+1}e_je_k)$ (or $i + 1 = j$). Symmetrically, we can define back-stable and forw-stable for e_j and e_k.

Note that back-stable plus forw-stable means stable. This applies Lemma 9.

▶ **Observation 11.** *Assume e_b is back-stable in $\triangle e_ae_be_c$. See Figure 2. Then,*

- *it is also back-stable in $\triangle e_{a+1}e_be_c$ when $a + 1 \neq b$ and $\triangle a_{a+1}e_be_c$ is finite.*
- *it is also back-stable in $\triangle e_ae_be_{c+1}$ when $c + 1 \neq a$ and $\triangle e_ae_be_{c+1}$ is finite.*

These claims are trivial; see an enhanced version with a proof in Appendix A (Observation 22).

▶ **Observation 12.** *Throughout Algorithm 1, the following hold.*

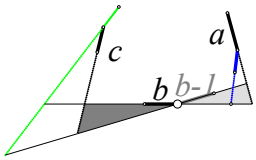

Figure 2 Illustration of Observation 11.

1. *$\triangle e_r e_s e_t$ has a finite area, which strictly decreases after every change of r, s, t.*
2. *Edges e_r, e_s, e_t are back-stable in $\triangle e_r e_s e_t$.*
3. *Edges e_s, e_t are forw-stable after the repeat-until sentence (Line 3 to Line 6), and e_r is forw-stable when the algorithm terminates.*

Proof. Part 1 is obvious. Part 3 is easy: Whenever one of e_r, e_s, e_t is not forw-stable, the algorithm moves it forwardly. We prove part 2 in the following. Initially, $\mathsf{Area}(\triangle e_{r+1} e_s e_t) < \mathsf{Area}(\triangle e_r e_s e_t)$, so $\mathsf{Area}(\triangle e_r e_s e_t) < \mathsf{Area}(\triangle e_{r-1} e_s e_t)$ by Lemma 9, i.e. e_r is back-stable. When $r \leftarrow r+1$ is to be executed at Line 10, $\mathsf{Area}(\triangle e_{r+1} e_s e_t) < \mathsf{Area}(\triangle e_r e_s e_t)$. This means e_r will be back-stable after this sentence. Furthermore, by Observation 11, a back-stable edge remains back-stable when we move another edge forwardly, so e_r remains back-stable when s or t is increased. By some similar arguments, e_s, e_t are always back-stable. Notice that initially e_s, e_t are back-stable since they are stable (guaranteed by the previous step). ◀

Algorithm 1 terminates eventually according to part 1 of Observation 12. Moreover, $\triangle e_r e_s e_t$ is 3-stable at the end. This follows from the other two parts of Observation 12.

Finally, observe that r can never return to 1 according to the fact that the initial triangle $\triangle e_r e_s e_t$ is the smallest one with $r = 1$. Moreover, observe that r, s, t can only move clockwise and e_r, e_s, e_t always lie in clockwise order. These together imply that the total number of changes of r, s, t is bounded by $O(n)$ and hence Algorithm 1 runs in $O(n)$ time. [1]

3 Compute all the 3-stable triangles in $O(n)$ time

Recall the framework of our algorithm in Subsection 1.2. This section presents the kernel of our algorithm — the killing criterion. First, we give some observations and a lemma.

▶ **Definition 13.** See Figure 3. Given rays r, r' originating at O and a hyperbola branch h admitting r, r' as asymptotes. Construct an arbitrary tangent line of h and assume that it intersects r, r' at points A, A' respectively. From basic knowledge of hyperbolas, the area of $\triangle OAA'$ is a constant. This area is defined as the *triangle-area* of h, denoted by $\mathsf{Area}(h)$.

▶ **Observation 14.** *Let r, r', O, h be the same as above and ϕ be the quadrant region bounded by r, r' and containing h. Consider any halfplane g which contains O and is delimited by l.*

1. *The area of $g \cap \phi$ is smaller than $\mathsf{Area}(h)$ if and only if l is disjoint with h.*
2. *The area of $g \cap \phi$ is identical to $\mathsf{Area}(h)$ if and only if l is tangent to h.*
3. *The area of $g \cap \phi$ is larger than $\mathsf{Area}(h)$ if and only if l cuts h (i.e. is a secant of h).*

[1] Although Algorithm 1 looks similar to the kernel step in [9] (by coincidence), our entire algorithm is essentially different from that in [9]. Most importantly, our first step for finding the "2-stable" triangle sets the initial value of (r, s, t) differently. In addition, our algorithm has an omitted subroutine symmetric to Algorithm 1 which handles the case where e_r is forw-stable but not back-stable, but [9] does not. Unfortunately, some previous reviewers irresponsibly regarded our algorithm in this section the same as the algorithm in [9] and claimed that this part of algorithm is not original.

Observation 14 is trivial; proof omitted. Recall that p_k is the half-plane delimited by ℓ_k and containing P for each k. Let $\phi_k^+ = \mathsf{p}_{k-1} \cap \mathsf{p}_k^C$ and $\phi_k^- = \mathsf{p}_{k-1}^C \cap \mathsf{p}_k$ denote two quadrant regions divided by ℓ_{k-1} and ℓ_k. (Subscripts are taken modulo n in all such places.)

▶ **Definition 15.** Consider any vertex v_k. See Figure 4.

- For every e_j such that $e_j \prec e_k$ and $e_j \neq e_{k-1}$, define $\mathsf{h}^+_{v_k,e_j}$ to be the hyperbola branch asymptotic to ℓ_{k-1}, ℓ_k in ϕ_k^+ with triangle-area as much as the area of $\phi_k^- \cap \mathsf{p}_j$.
- For every e_j such that $e_{k-1} \prec e_j$ and $e_j \neq e_k$, define $\mathsf{h}^-_{v_k,e_j}$ to be the hyperbola branch asymptotic to ℓ_{k-1}, ℓ_k in ϕ_k^- with triangle-area as much as the area of $\phi_k^+ \cap \mathsf{p}_j$.

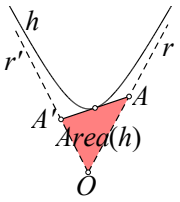

Figure 3 Illustration of Definition 13

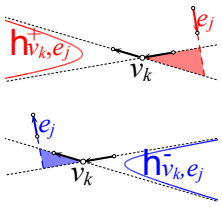

Figure 4 Illustration of Definition 15

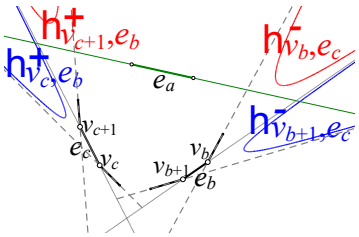

Figure 5 Illustration of Observation 16

For convenience, we use two abbreviations in the following. Let "intersect" be short for "cut or be tangent to" and "avoid" be short for "be disjoint with or tangent to".

▶ **Observation 16.** *Assume $\triangle e_a e_b e_c$ has a finite area where e_b, e_c are stable. See Figure 5.*

1. *ℓ_a intersects $\mathsf{h}^-_{v_{b+1},e_c}$ when $b+1 \neq c$ and avoids $\mathsf{h}^-_{v_b,e_c}$ when $e_{b-1} \prec e_c$.*
2. *ℓ_a intersects $\mathsf{h}^+_{v_c,e_b}$ when $b \neq c-1$ and avoids $\mathsf{h}^+_{v_{c+1},e_b}$ when $e_b \prec e_{c+1}$.*

***Note**: Applying Observation 2 on $\triangle e_a e_b e_c$, we get $e_b \prec e_c$. Therefore, each of the four hyperbolas $\mathsf{h}^-_{v_{b+1},e_c}$, $\mathsf{h}^-_{v_b,e_c}$, $\mathsf{h}^+_{v_c,e_b}$, $\mathsf{h}^+_{v_{c+1},e_b}$ are defined under the corresponding condition.*

Proof. Assume $e_{b+1} \prec e_c$. Because e_b is stable, $\mathsf{Area}(\triangle e_a e_b e_c) \leq \mathsf{Area}(\triangle e_a e_{b+1} e_c)$. So the area of $\mathsf{p}_a \cap (\mathsf{p}_b^C \cap \mathsf{p}_{b+1})$ is at least the area of $\mathsf{p}_c \cap \mathsf{p}_b \cap \mathsf{p}_{b+1}^C$. Namely, the former area is no smaller than $\mathsf{Area}(\mathsf{h}^-_{v_{b+1},e_c})$. Applying Observation 14, this means ℓ_a intersects $\mathsf{h}^-_{v_{b+1},e_c}$.

Assume $e_{b-1} \prec e_c$. This implies that $e_a \neq e_{b-1}$; otherwise $e_a \prec e_c$ and $\triangle e_a e_b e_c$ is infinite. Because $e_{b-1} \neq e_a$ and e_b is stable, $\mathsf{Area}(\triangle e_a e_b e_c) \leq \mathsf{Area}(\triangle e_a e_{b-1} e_c)$. Therefore, the area of $\mathsf{p}_a \cap (\mathsf{p}_{b-1}^C \cap \mathsf{p}_b)$ is at most the area of $\mathsf{p}_c \cap \mathsf{p}_{b-1} \cap \mathsf{p}_b^C$. In other words, the former area is no larger than $\mathsf{Area}(\mathsf{h}^-_{v_b,e_c})$. This means ℓ_a avoids $\mathsf{h}^-_{v_b,e_c}$ by Observation 14.

Symmetrically, because e_c is stable, we can prove claim 2. ◀

Recall the 3-stable triangle $\triangle e_r e_s e_t$ and the conditions (1) and (2) in Subsection 1.2. Observation 16 represents "stable" by line-hyperbola intersection conditions. The following lemma provides sufficient conditions of (1) and (2) in guise of line-hyperbola intersections.

▶ **Lemma 17.** *Assume $e_b \in \{e_s, \ldots, e_t\}$, $e_c \in \{e_t, \ldots, e_r\}$, $e_b, e_{b+1}, e_c, e_{c+1}$ are distinct and $e_b \prec e_{c+1}$. Note that $h^+_{v_{c+1},e_b}$, $\mathsf{h}^-_{v_{b+1},e_{c+1}}$, $h^+_{v_{c+1},e_{b+1}}$ and $\mathsf{h}^-_{v_{b+1},e_c}$ are defined; see Figure 6.*

1. ***a.** When some edge pair $(e_b, e_{c'}) \in \{(e_b, e_{c+1}), (e_b, e_{c+2}), \ldots, (e_b, e_r)\}$ is not dead, and hence there exists e_a so that $\triangle e_a e_b e_{c'}$ is 3-stable,*

   ℓ_a must (i) intersect both $h^+_{v_{c+1},e_b}$ and $\mathsf{h}^-_{v_{b+1},e_{c+1}}$ and (ii) belongs to $\{\ell_{c+2}, \ldots, \ell_{b-1}\}$.

   ***b.** If (I) no line in $\{\ell_{c+2}, \ldots, \ell_{b-1}\}$ intersects both $h^+_{v_{c+1},e_b}$ and $\mathsf{h}^-_{v_{b+1},e_{c+1}}$, we can infer that $(e_b, e_{c+1}), (e_b, e_{c+2}), \ldots, (e_b, e_r)$ are all dead, namely, (1) holds.*

To be clear, throughout this paper, $(e_b, e_{c+1}), (e_b, e_{c+2}), \ldots, (e_b, e_r)$ is empty when $c = r$.

2. **a.** *When some edge pair* $(e_{b'}, e_c) \in \{(e_{b+1}, e_c), (e_{b+2}, e_c), \ldots, (e_t, e_c)\}$ *is not dead, and hence there exists* e_a *so that* $\triangle e_a e_{b'} e_c$ *is 3-stable,*

   ℓ_a *must (i) avoid both* $h^+_{v_{c+1}, e_{b+1}}$ *and* $\mathsf{h}^-_{v_{b+1}, e_c}$ *and (ii) belongs to* $\{\ell_{c+2}, \ldots, \ell_{b-1}\}$.

   b. *If (II) no line in* $\{\ell_{c+2}, \ldots, \ell_{b-1}\}$ *avoids both* $h^+_{v_{c+1}, e_{b+1}}$ *and* $\mathsf{h}^-_{v_{b+1}, e_c}$, *we can infer that* $(e_{b+1}, e_c), (e_{b+2}, e_c), \ldots, (e_t, e_c)$ *are all dead, namely, (2) holds.*

   To be clear, throughout this paper, $(e_{b+1}, e_c), (e_{b+2}, e_c), \ldots, (e_t, e_c)$ is empty when $b = t$.

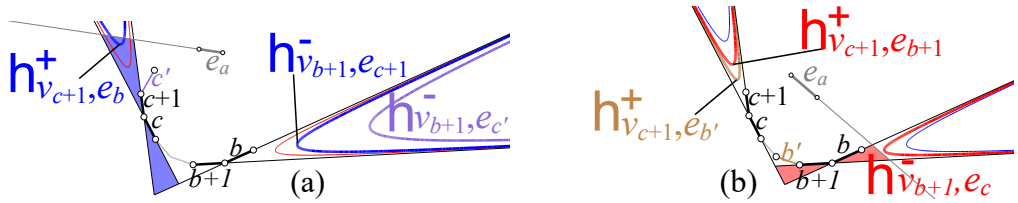

Figure 6 Illustration of the proof of Lemma 17.

Proof. 1.b and 2.b are the contrapositives of 1.a and 2.a; so we only prove 1.a and 2.a. See Figure 6 (a) and Figure 6 (b) for the illustrations of the proofs of 1.a and 2.a respectively.

Proof of 1.a-(i). Because $e_{c'} \in \{e_{c+1}, \ldots, e_r\}$ and is stable in $\triangle e_a e_b e_{c'}$, by the unimodality in Lemma 9, $\mathsf{Area}(\triangle e_a e_b e_{c+1}) \leq \mathsf{Area}(\triangle e_a e_b e_c)$. Equivalently, the area of $\mathsf{p}_a \cap (\mathsf{p}_c \cap \mathsf{p}^C_{c+1})$ is at least $\mathsf{Area}(h^+_{v_{c+1}, e_b})$. Applying Observation 14, this means ℓ_a intersects $h^+_{v_{c+1}, e_b}$.

Applying Observation 16.1 on $\triangle e_a e_b e_{c'}$, line ℓ_a intersects $\mathsf{h}^-_{v_{b+1}, e_{c'}}$. Moreover, $\mathsf{h}^-_{v_{b+1}, e_{c'}}$ is clearly contained in the area bounded by $\mathsf{h}^-_{v_{b+1}, e_{c+1}}$. Therefore, ℓ_a intersects $\mathsf{h}^-_{v_{b+1}, e_{c+1}}$.

Proof of 2.a-(ii). Because $\triangle e_a e_b e_{c'}$ is defined, $\ell_a \in \{\ell_{c'+1}, \ldots, \ell_{b-1}\}$, which implies (ii).

Proof of 2.a-(i). Because $e_{b'} \in \{e_{b+1}, \ldots, e_t\}$ and is stable in $\triangle e_a e_{b'} e_c$, by the unimodality in Lemma 9, $\mathsf{Area}(\triangle e_a e_{b+1} e_c) \leq \mathsf{Area}(\triangle e_a e_b e_c)$. Equivalently, the area of $\mathsf{p}_a \cap (\mathsf{p}^C_b \cap \mathsf{p}_{b+1})$ is at most $\mathsf{Area}(\mathsf{h}^-_{v_{b+1}, e_c})$. Applying Observation 14, this means ℓ_a avoids $\mathsf{h}^-_{v_{b+1}, e_c}$.

Applying Observation 16.2 on $\triangle e_a e_{b'} e_c$, line ℓ_a avoids $\mathsf{h}^+_{v_{c+1}, e_{b'}}$. Moreover, the area bounded by $\mathsf{h}^+_{v_{c+1}, e_{b'}}$ clearly contains $h^+_{v_{c+1}, e_{b+1}}$. Therefore, ℓ_a avoids $h^+_{v_{c+1}, e_{b+1}}$.

Proof of 2.a-(ii). Because $\triangle e_a e_{b'} e_c$ is defined, $\ell_a \in \{\ell_{c+1}, \ldots, \ell_{b'-1}\}$. Since this triangle is 3-stable, $e_c \prec e_a$. However, edges in $e_b, \ldots, e_{b'-1}$ are chasing e_c, so they do not contain e_a. So, $\ell_a \in \{\ell_{c+1}, \ldots, \ell_{b-1}\}$. Because $e_b \prec e_{c+1}$, we have $e_{b'} \prec e_{c+1}$. However, $e_a \prec e_{b'}$ because $\triangle e_a e_{b'} e_c$ is 3-stable. So, $a \neq c+1$. Altogether, $\ell_a \in \{\ell_{c+2}, \ldots, \ell_{b-1}\}$, i.e. (ii) holds. ◀

To design a killing criterion as mentioned in Subsection 1.2, we are looking for a condition such that first it is easy to compute, and second itself and its negative implies (1) and (2) respectively. In Lemma 17, we give two sufficient conditions of (1) and (2), which are (I) and (II) respectively, and thus reduce the problem to find an easy-to-compute condition who and whose negative imply (I) and (II). We design such a condition (X) in the next lemma.

The assumption of b, c henceforth follows Lemma 17 unless otherwise stated.

Notation. Let $G^+_{b,c}, G^-_{b,c}, H^+_{b,c}, H^-_{b,c}$ denote $h^+_{v_{c+1}, e_b}, \mathsf{h}^-_{v_{b+1}, e_{c+1}}, h^+_{v_{c+1}, e_{b+1}}, \mathsf{h}^-_{v_{b+1}, e_c}$ for short. Denote by $L^{GG}_{b,c}$ the common tangent of $G^+_{b,c}$ and $G^-_{b,c}$, and denote the other three common tangents by $L^{HG}_{b,c}, L^{GH}_{b,c}$ and $L^{HH}_{b,c}$ correspondingly; see Figure 7 and 8. Omit subscripts b, c when they are clear in context. Assume these four common tangents are directed; the direction of such a tangent is from its intersection with ℓ_{c+1} to its intersection with ℓ_b.

▶ **Lemma 18.** *See Figure 7. Choose an arbitrary directed line L going from a point in (open) segment $\overline{(L^{GG} \cap \ell_{c+1})(L^{HH} \cap \ell_{c+1})}$ to a point in (open) segment $\overline{(L^{GG} \cap \ell_b)(L^{HH} \cap \ell_b)}$. If (X) P lies on the right of L, we have (I): no line $\ell_a \in \{\ell_{c+2}, \dots, \ell_{b-1}\}$ intersects both $G^+_{b,c}$ and $G^-_{b,c}$. Otherwise, we have (II): no line $\ell_a \in \{\ell_{c+2}, \dots, \ell_{b-1}\}$ avoids both $H^+_{b,c}$ and $H^-_{b,c}$.*

Proof. We state two crucial observations.

(i) If (I) fails, a point of P lies in or on the left of L^{GG}.

(ii) If (II) fails, all points of P lie in or on the right of L^{HH}.

Proof of (i). Assume (I) fails, so there is $\ell_a \in \{\ell_{c+2}, \dots, \ell_{b-1}\}$ that intersects $G^+_{b,c}, G^-_{b,c}$. This means that a point in e_a (and hence in P) lies in or on the left of L^{GG}.

Proof of (ii). Assume (II) fails, so there is $\ell_a \in \{\ell_{c+2}, \dots, \ell_{b-1}\}$ which avoids $H^+_{b,c}$ and $H^-_{b,c}$. This implies that P lies in the area containing the right of L^{HH} and L^{HH}.

If P lies on the right of L, no point in P lies in or on the left of L^{GG}, so (I) holds according to (i). Otherwise, a point of P would lie on the left of L^{HH}, so (II) holds applying (ii). ◀

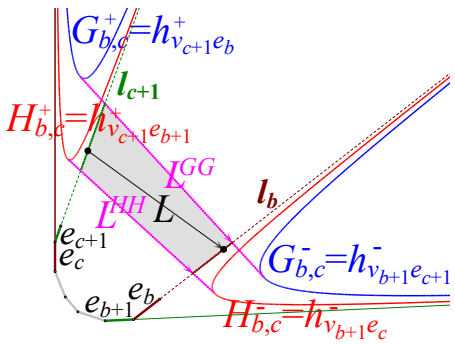

Figure 7 L^{GG}, L^{HH} and Lemma 18.

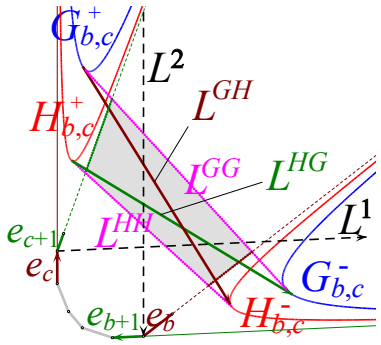

Figure 8 L^{HG}, L^{GH} and Observation 19.

We can now briefly outline the Rotate-and-Kill process based on criterion (X). In each iteration, after *compute* $\mathsf{a}_{b,c}$ and *output* $\triangle \mathsf{a}_{b,c}e_be_c$ (as a candidate of a 3-stable triangle), we proceed to the next iteration by either killing b or killing c. To select the right one to kill, we *choose a line* L and compute (X), i.e. *compute whether P lies on the right of L or not.* Note that this process is not finalized because we have not specified how to choose L.

Computing (X) takes $O(\log n)$ time since P is convex, or more satisfactory, amortized $O(1)$ time if the slope of L changes monotonously throughout the process. Therefore, toward a linear time algorithm, the key is to choose L so that its slope changes monotonously.

This is not easy. For example, if we choose L to be a particular line (like $L^{GG}_{b,c}$ or $L^{HH}_{b,c}$) at every iteration, the slope is not monotone; see counterexamples in Appendix B. Nevertheless, by choosing L more deliberately as shown below, we obtain the monotonicity as required.

For a directed line L, let $d(L)$ denote its direction, which is an angle in $[0, 2\pi)$, adopting the convention that $d(\overrightarrow{OA})$ increases when A rotates clockwise around O. Let d_1, d_2 be the opposite directions of e_{s+1}, e_r. Without loss of generality, assume that $[d_1, d_2] \subset [0, 2\pi)$.

▶ **Observation 19.** *Let (e_b, e_c) be any pair following the assumption of Lemma 17.*

1. $[d(L^{HG}_{b,c}), d(L^{GH}_{b,c})] \subset [d_1, d_2]$.

2. For $d \in [d(L^{HG}_{b,c}), d(L^{GH}_{b,c})]$, we can compute in constant time a line L with direction d from a point in (open segment) $\overline{(L^{GG} \cap \ell_{c+1})(L^{HH} \cap \ell_{c+1})}$ to a point in (open segment) $\overline{(L^{GG} \cap \ell_b)(L^{HH} \cap \ell_b)}$.

Proof. See Figure 8. Let $L^1_{b,c}$ be the line at v_{c+1} with the opposite direction to e_{b+1}, and $L^2_{b,c}$ be the line at v_{b+1} with the opposite direction to e_c. Note that the area bounded by L^1, ℓ_b, ℓ_{b+1} is infinite, whereas $G^-_{b,c}$ has a finite triangle-area. So L^1 intersects $G^-_{b,c}$ by Observation 14. Thus L^1 intersects $G^-_{b,c}$ and avoids $H^+_{b,c}$. Similarly, L^2 intersect $G^+_{b,c}$ and avoids $H^-_{b,c}$. These together imply that $[d(L^{HG}_{b,c}), d(L^{GH}_{b,c})] \subset [d(L^1_{b,c}), d(L^2_{b,c})]$, which further implies Claim 1 because $d(L^1_{s,t}) = d_1$ whereas $d(L^2_{t,r}) = d_2$.

L^{HG} and L^{GH} clearly satisfy the requirement of L in Lemma 18. This implies part 2. ◀

We present our final killing criterion (with the specification of L) in Algorithm 2.

1 **If** $(b+1=c)$ Return c;
2 **If** (e_b is not chasing e_{c+1}) Return b;
Note: $e_b, e_{b+1}, e_c, e_{c+1}$ are now four distinct edges and $e_b \prec e_{c+1}$.
3 Select some direction $d \in [d(L^{HG}_{b,c}), d(L^{GH}_{b,c})]$ (specified in (3) below);
4 Find any line L in the gray area with direction d (using Observation 19.2);
5 Compute the supporting line L' of P with direction d so that P is on the right of L'.
6 Compare L' with L and thus determine (X). (If L' lies on the right of L, so does P, and we determine (X) holds; otherwise, P must intersect L, and we determine (X) fails.)
7 **If** (X) return b; **Else** return c.

Algorithm 2: Criterion for killing b, c.

Selection of d. We apply the following equation in Line 3 in choosing d:

$$d_{\mathsf{this-iteration}} = \begin{cases} d_{\mathsf{previous-iteration}}, & d_{\mathsf{previous-iteration}} \in [d(L^{HG}_{b,c}), d(L^{GH}_{b,c})]; \\ d(L^{HG}_{b,c}), & \text{otherwise.} \end{cases} \tag{3}$$

Correctness. If $b+1=c$, no edge in $e_{b+1}, \ldots, e_t$ can chase e_c, so we can kill c. If e_b is not chasing e_{c+1}, edge e_b cannot chase any edge in $e_{c+1}, \ldots, e_r$, so we can kill b. If (X), condition (I) holds by Lemma 18 and thus (1) holds by Lemma 17.1; so we can kill b. Otherwise, (II) holds by Lemma 18 and thus (2) holds by Lemma 17.2; so we can kill c.

Running time. By the following lemma, we get the monotonicity of the slope mentioned above. So it takes amortized $O(1)$ time to compute L'. The other steps cost $O(1)$ time.

▶ **Lemma 20.** *Assume we were given (b,c) in some iteration and (b',c') in a later iteration, where (b,c) and (b',c') both satisfy the assumption in Lemma 17, then $d(L^{GH}_{b',c'}) \geq d(L^{HG}_{b,c})$. As a corollary, when (3) is applied, variable d increases monotonously during the algorithm.*

Proof. We first prove the corollary part from the first part (see an illustration in Appendix B). We want to show $d_{\mathsf{previous-iteration}} \leq d_{\mathsf{this-iteration}}$. This trivially holds when $d_{\mathsf{previous-iteration}} \in [d(L^{HG}_{b,c}), d(L^{GH}_{b,c})]$. Consider the other case. Assume without loss of generality $d_{\mathsf{previous-iteration}} = d(L^{HG}_{b^*,c^*})$ where (b^*, c^*) denotes some previous iteration. We have (i) $d(L^{HG}_{b^*,c^*}) \leq d(L^{GH}_{b,c})$ (according to the first part of the lemma) and (ii) $d(L^{HG}_{b^*,c^*}) = d_{\mathsf{previous-iteration}} \notin [d(L^{HG}_{b,c}), d(L^{GH}_{b,c})]$ (by assumption). Together, $d(L^{HG}_{b^*,c^*}) < d(L^{HG}_{b,c})$. In other words, $d_{\mathsf{previous-iteration}} < d_{\mathsf{this-iteration}}$. In either way, d increases (non-strictly).

We prove the first part in the following. Let $A = \ell_{b+1} \cap \ell_{c+1}, B = \ell_{b'} \cap \ell_{c'}$.

First, consider the case where $b' > b$ and $c' > c$. See Figure 9. Denote

$$M^- = \begin{cases} v_{b'+1}, & b' = b+1; \\ \ell_{b+1} \cap \ell_{b'}, & b' \geq b+2 \end{cases} \text{ and } M^+ = \begin{cases} v_{c'+1}, & c' = c+1; \\ \ell_{c+1} \cap \ell_{c'}, & c' \geq c+2. \end{cases}$$

Denote the reflections of A around M^-, M^+ by A^-, A^+ respectively. Denote the reflections of B around M^-, M^+ by B^-, B^+ respectively. Let $L = \overrightarrow{A^+A^-}$ and $L' = \overrightarrow{B^+B^-}$.

We state three equalities or inequalities which together imply the key inequality.

(i) $d(L^{HG}_{b,c}) \le d(L)$. (ii) $d(L) = d(\overrightarrow{M^+M^-}) = d(L')$. (iii) $d(L') \le d(L^{GH}_{b',c'})$.

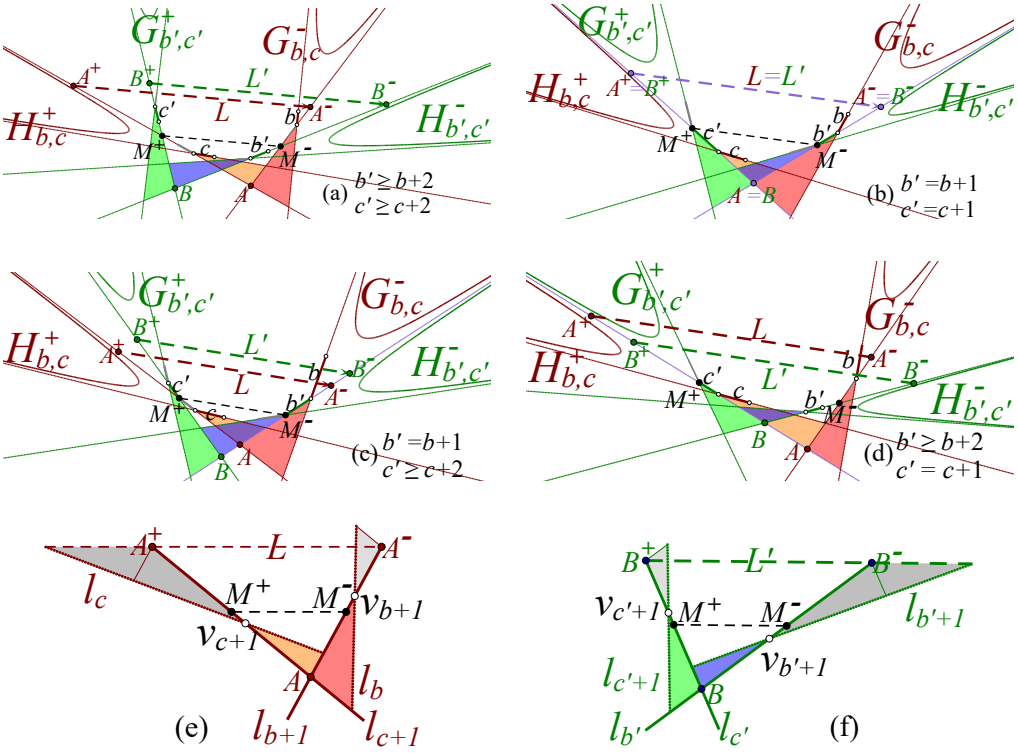

Figure 9 Proof of Lemma 20 - part I.

Proof of (i): This reduces to showing that (i.1) L intersects $H^+_{b,c}$ and (i.2) L avoids $G^-_{b,c}$. Now, let us focus on the objects shown in Figure 9 (e). Notice that A, v_{c+1}, M^+, A^+ lie in this order in line ℓ_{c+1}. Further since $|AM^+| = |A^+M^+|$, we know $|A^+v_{c+1}| > |Av_{c+1}|$. Thus the area of $h \cap (\mathsf{p}_c \cap \mathsf{p}^C_{c+1})$, where h denotes the half-plane parallel to l_{b+1} that admits A^+ on its boundary and contains v_{c+1}, is larger than the area of the (yellow) triangle $\mathsf{p}_{b+1} \cap \mathsf{p}^C_c \cap \mathsf{p}_{c+1}$, which equals $\mathsf{Area}(h^+_{v_{c+1},e_{b+1}})$. So the triangle area bounded by l_c, l_{c+1} and L is even larger than $\mathsf{Area}(h^+_{v_{c+1},e_{b+1}})$. By Observation 14, this means L intersects $h^+_{v_{c+1},e_{b+1}}$, i.e. (i.1) holds.

Assume A, M^-, v_{b+1}, A^- lie in this order in ℓ_{b+1} (otherwise the order would be A, M^-, A^-, v_{b+1}, which is easier). Similarly, the area bounded by l_b, l_{b+1} and L is smaller than $\mathsf{Area}(h^-_{v_{b+1},e_{c+1}})$, which by Observation 14 means that L avoids $h^-_{v_{b+1},e_{c+1}}$, i.e. (i.2) holds.

Proof of (iii): This reduces to showing that (iii.1) L' intersects $H^-_{b',c'}$ and (iii.2) L' avoids $G^+_{b',c'}$. See Figure 9 (f); they are symmetric to (i.1) and (i.2) respectively; proof omitted.

In the following, assume $b = b'$ or $c = c'$. We discuss four subcases.

Case 1 $b' = b+1, c' = c$. See Figure 10 (a). Note that $H^+_{b,c} = G^+_{b',c'}$ in this case. Let A^- be the reflection of A around v_{b+1} and B^- the reflection of B around $v_{b'+1}$. Let L be the tangent line of $H^+_{b,c}$ that passes through A^-, and L' the tangent line of $G^+_{b',c'}$ that passes through B^-. We argue that (i) $d(L^{HG}_{b,c}) \le d(L)$ and (iii) $d(L') \le d(L^{GH}_{b',c'})$ still hold in this case. They follow from the observations that the triangles with light color in the figure are smaller than their opposite triangles with dark color, which follow from the

facts $|Bv_{b'+1}| = |B^-v_{b'+1}|$ and $|Av_{b+1}| = |A^-v_{b+1}|$. Moreover, points $v_{b'+1}, v_{b+1}, B^-, A^-$ clearly lie in this order in ℓ_{b+1}, and thus $d(L) < d(L')$. Altogether, $d(L^{HG}_{b,c}) \le d(L^{GH}_{b',c'})$.

Case 2 $b' = b, c' = c+1$. See Figure 10 (b); symmetric to Case 1; proof omitted.

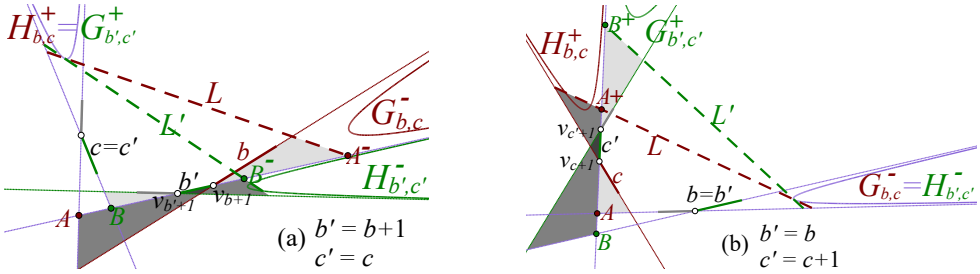

Figure 10 Proof of Lemma 20 - part II.

Case 3 $b' \ge b+2, c' = c$. **Hint: This case is more difficult because $G^+_{b',c'}$ is now "below" $H^+_{b,c}$ as shown in Figure 11 (a); this proof contains several more tricks.** Let A^-, B^- be the reflection of A, B around M^- respectively and L be the tangent line of $H^+_{b,c}$ that passes through A^-, and L' the tangent line of $G^+_{b',c'}$ that passes through B^-. As in the previous cases, (i) and (iii) hold and thus it reduces to showing that $d(L') > d(L)$.

See Figure 11 (b). Make a parallel line L_2 of L at B^-, which intersects ℓ_c, ℓ_{c+1} at B_1, B_2 respectively. It reduces to showing that the area bounded by L_2, l_c, l_{c+1}, namely $\mathsf{Area}(\triangle v_{c+1}B_1B_2)$, is smaller than the area bounded by L', l_c, l_{c+1}. The latter equals to $\mathsf{Area}(G^+_{b',c'}) = \mathsf{Area}(\triangle v_{c+1}BX)$, where $X = \ell_{c+1} \cap \ell_{b'}$. Let X^- be the reflection of X around M^-. Assume the parallel line of L at X^- intersects ℓ_c, ℓ_{c+1} at X_1, X_2 respectively. Let $D = \ell_{b+1} \cap \ell_c$, and E be the point on ℓ_c so that $\overline{XE}$ is parallel to $\overline{AD}$. Clearly, $\mathsf{Area}(\triangle v_{c+1}B_1B_2) < \mathsf{Area}(\triangle v_{c+1}X_1X_2)$ and $\mathsf{Area}(\triangle v_{c+1}BX) > \mathsf{Area}(\triangle v_{c+1}EX)$. So it further reduces to proving that (I) $\mathsf{Area}(\triangle v_{c+1}X_1X_2) < \mathsf{Area}(\triangle v_{c+1}EX)$.

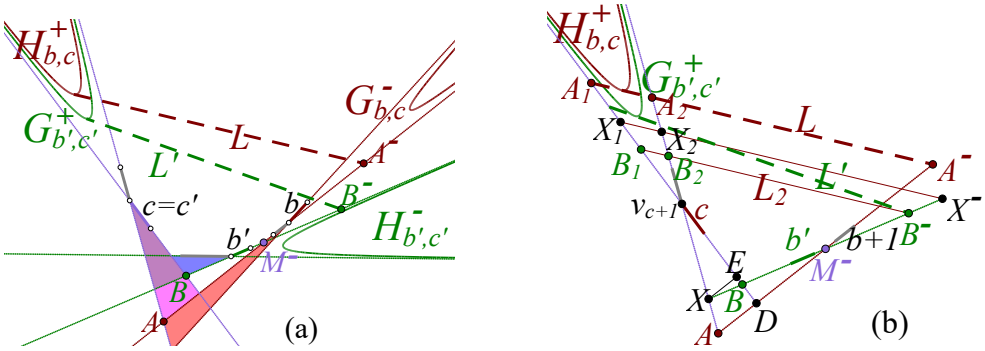

Figure 11 Proof of Lemma 20 - part III.

Assume L intersects ℓ_c, ℓ_{c+1} at A_1, A_2. We know (a) $\mathsf{Area}(\triangle v_{c+1}A_1A_2) = \mathsf{Area}(\triangle v_{c+1}DA)$ since L is tangent to $H^+_{b,c}$, and so (b): $|v_{c+1}A_2| < |v_{c+1}A|$. Since segment $\overline{A^-X^-}$ is a translate of $\overline{XA}$, (c) $|AX| = |A_2X_2|$. Combining (b) with (c), $|A_2X_2|/|v_{c+1}A_2| > |AX|/|v_{c+1}A|$, so (d) $|v_{c+1}X_2|/|v_{c+1}A_2| < |v_{c+1}X|/|v_{c+1}A|$. Further since $\overline{X_1X_2}$ is parallel to $\overline{A_1A_2}$ whereas $\overline{XE}$ is parallel to $\overline{AD}$, fact (a) and (d) together imply (I).

Case 4 $b' = b, c' \ge c+2$. Symmetric to Case 3. For completeness, we prove it in Appendix B.

◀

A Proofs of basic observations and lemmas

Lemma 6 states that the 3-stable triangles are pairwise interleaving. As its corollary, Corollary 7 states that there are only $O(n)$ 3-stable triangles.

Proof of Lemma 6. Suppose that $\triangle e_i e_j e_k, \triangle e_r e_s e_t$ are two 3-stable triangles which are not interleaving. Depending on the number of edges these two triangles share, there are three cases. First, if they share two common edges, it is trivial that they are interleaving.

Second, assume they share one common edge, e.g., $e_j = e_s$. Because not interleaving, we can assume that $e_k, e_t, e_r, e_i, e_j = e_s$ are in clockwise order as shown in Figure 12 (a), otherwise $e_t, e_k, e_i, e_r, e_j = e_s$ are in clockwise order and it is symmetric. Let I be the intersection of ℓ_i and ℓ_r. Let B, C, E, B', C', E' be the intersections as shown in the figure. Because triangle $\triangle e_i e_j e_k$ is 3-stable, $\mathsf{Area}(\triangle e_i e_j e_k) \leq \mathsf{Area}(\triangle e_r e_j e_k)$. In other words, $\mathsf{Area}(\triangle ICC') \leq \mathsf{Area}(\triangle IBB')$. However, $\mathsf{Area}(\triangle ICC') > \mathsf{Area}(\triangle IEE)$. Together, $\mathsf{Area}(\triangle IEE') < \mathsf{Area}(\triangle IBB')$. Equivalently, $\mathsf{Area}(\triangle e_i e_s e_t) < \mathsf{Area}(\triangle e_r e_s e_t)$, which means e_r is not stable in $\triangle e_r e_s e_t$. This contradicts the assumption that $\triangle e_r e_s e_t$ is 3-stable.

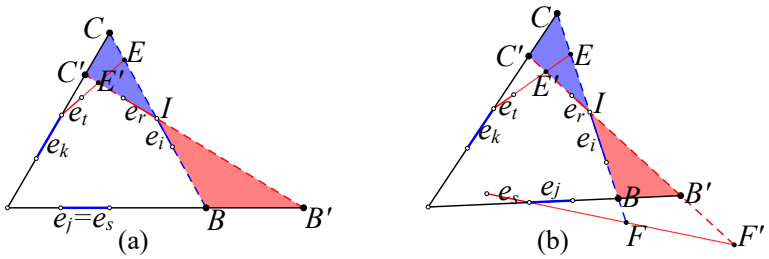

Figure 12 If two triangles are not interleaving, they cannot be both 3-stable.

Last, consider the case of no common edges. Recall the notation $(X \circlearrowright X')$ in Section 2 which indicates a boundary portion of P with endpoints X, X'. Because not interleaving, among $(v_{j+1} \circlearrowright v_k), (v_{k+1} \circlearrowright v_i)$ and $(v_{i+1} \circlearrowright v_j)$, there must be one boundary portion that contains no edge from $\{e_r, e_s, e_t\}$, whereas the others respectively contain two and one. (Note that e_r, e_s, e_t cannot be contained in the same portion. Otherwise $\triangle e_r e_s e_t$ is unbounded and hence not 3-stable.) Without loss of generality, assume that the mentioned three portions respectively contain $\{e_s\}$, $\{e_t, e_r\}$ and $\varnothing$, as shown in Figure 12 (b). The following proof is similar to the previous case. Let $B, C, E, F, B', C', E', F'$ be the intersections as shown in the figure. We have $\mathsf{Area}(\triangle e_i e_j e_k) \leq \mathsf{Area}(\triangle e_r e_j e_k)$ and so $\mathsf{Area}(\triangle ICC') \leq \mathsf{Area}(\triangle IBB')$, thus $\mathsf{Area}(\triangle IEE') < \mathsf{Area}(\triangle IFF')$ and so $\mathsf{Area}(\triangle e_i e_s e_t) < \mathsf{Area}(\triangle e_r e_s e_t)$, which means e_r is not stable in $\triangle e_r e_s e_t$, which contradicts the 3-stable assumption of $\triangle e_r e_s e_t$. ◀

Proof of Corollary 7. Assume the number of 3-stable triangles is m. By Lemma 6, we can label the 3-stable triangles by $\triangle a_1 b_1 c_1, \ldots, \triangle a_m b_m c_m$ so that $a_1, \ldots, a_m, b_1, \ldots, b_m, c_1, \ldots, c_m$ lie in clockwise order (in the non-strictly manner as in Definition 5). For each i $(1 \leq i \leq m)$, denote δ_i to be $|\{a_1, \ldots, a_i\}| + |\{b_1, \ldots, b_i\}| + |\{c_1, \ldots, c_i\}|$, where $|\cdot|$ indicates the size of the set here. We know $3 \leq \delta_1 < \delta_2 < \ldots < \delta_m < n + 3$ and this means that $m \leq n$.

Alternatively, by applying Lemma 6, Subsection 1.2 has shown that the number of pairs (e_b, e_c) that are not dead is only $O(n)$, and this simply implies that $m = O(n)$. ◀

Lemma 9 states the unimodality of $\mathsf{Area}(\triangle e_a e_b e_c) \mid e_a$ for fixed b, c.

Proof of Lemma 9. For every (i, j), let $I_{i,j}$ denote the intersection between ℓ_i and ℓ_j. Assume $e_b \prec e_c$ and $\mathsf{D}_b \neq \mathsf{D}_c$. We classify the edges in $(\mathsf{D}_b \circlearrowright \mathsf{D}_c)$ into two categories: e_a is

negative if $|I_{a,c}v_{a+1}| \le |I_{a,b}v_{a+1}|$ and *positive* otherwise; see the top pictures in Figure 13. This lemma follows from the following three observations.

(i) If e_a is positive, its next edge e_{a+1} must also be positive. Therefore, when e_a is enumerated clockwise in $(\mathsf{D}_b \circlearrowright \mathsf{D}_c)$, we get several negative edges followed by several positive edges.
(ii) If e_a, e_{a+1} are both negative, $\mathsf{Area}(\triangle e_{a+1}e_be_c) < \mathsf{Area}(\triangle e_ae_be_c)$.
(iii) If e_a, e_{a+1} are both positive, $\mathsf{Area}(\triangle e_{a+1}e_be_c) > \mathsf{Area}(\triangle e_ae_be_c)$.

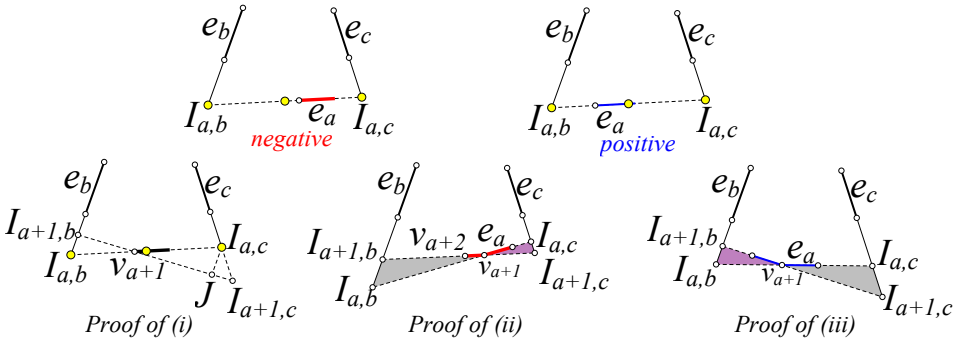

Figure 13 Illustration of the proof of Lemma 9

Proof of (i): Make a line at $I_{a,c}$ parallel to ℓ_b and assume it intersects ℓ_{a+1} at J. Since e_a is positive, $|I_{a,c}v_{a+1}| > |I_{a,b}v_{a+1}|$. It implies that $|Jv_{a+1}| > |I_{a+1,b}v_{a+1}|$. Therefore, $|I_{a+1,c}v_{a+1}| > |I_{a+1,b}v_{a+1}|$. Furthermore, $|I_{a+1,c}v_{a+2}| > |I_{a+1,b}v_{a+2}|$, i.e., e_{a+1} is positive.

Proof of (ii): Because e_a, e_{a+1} are negative, $|I_{a,c}v_{a+1}| \le |I_{a,b}v_{a+1}|$ and $|I_{a+1,c}v_{a+1}| < |I_{a+1,b}v_{a+1}|$. Therefore, $|I_{a,c}v_{a+1}| \cdot |I_{a+1,c}v_{a+1}| < |I_{a,b}v_{a+1}| \cdot |I_{a+1,b}v_{a+1}|$. In other words, $\triangle v_{a+1}I_{a,c}I_{a+1,c}$ is smaller than $\triangle v_{a+1}I_{a,b}I_{a+1,b}$, i.e., $\mathsf{Area}(\triangle e_{a+1}e_be_c) < \mathsf{Area}(\triangle e_ae_be_c)$.

Proof of (iii): Because e_a is positive, $|I_{a,c}v_{a+1}| > |I_{a,b}v_{a+1}|$ and $|I_{a+1,c}v_{a+1}| > |I_{a+1,b}v_{a+1}|$ (use the proof of (i)). Therefore, $|I_{a,c}v_{a+1}| \cdot |I_{a+1,c}v_{a+1}| > |I_{a,b}v_{a+1}| \cdot |I_{a+1,b}v_{a+1}|$, i.e. $\triangle v_{a+1}I_{a,c}I_{a+1,c}$ is larger than $\triangle v_{a+1}I_{a,b}I_{a+1,b}$, i.e., $\mathsf{Area}(\triangle e_{a+1}e_be_c) > \mathsf{Area}(\triangle e_ae_be_c)$.

Note that when e_a is negative and e_{a+1} is positive, the relation between $\mathsf{Area}(\triangle e_{a+1}e_be_c)$ and $\mathsf{Area}(\triangle e_ae_be_c)$ is undecided, and it may be equal. Anyway, Lemma 9 still holds. ◀

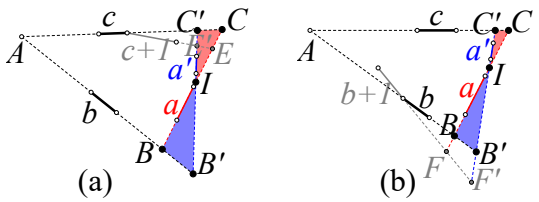

Figure 14 Illustration of the proof of the bi-monotonicity of $\{\mathsf{a}_{b,c}\}$.

Lemma 10 states the bi-monotonicity of $\{\mathsf{a}_{b,c}\}$.

Proof of Lemma 10. 1. First, assume $\mathsf{D}_b = \mathsf{D}_c$. See Figure 1 (b). We know $\mathsf{a}_{b,c}$ is the previous edge of D_b. If $\mathsf{D}_{c+1} = \mathsf{D}_b$, edge $\mathsf{a}_{b,c+1}$ equals the previous edge of D_b and hence equals $\mathsf{a}_{b,c}$. Otherwise $\mathsf{a}_{b,c+1}$ lies in $(\mathsf{D}_b \circlearrowright \mathsf{D}_{c+1})$ and hence lies after $\mathsf{a}_{b,c}$ (clockwise) in E.

Next, assume $\mathsf{D}_b \neq \mathsf{D}_c$. See Figure 14 (a). Let $e_a = \mathsf{a}_{b,c}$. We shall prove that $\mathsf{a}_{b,c+1} \neq e_{a'}$ for any $e_{a'}$ in $(\mathsf{D}_b \circlearrowright v_a)$. Assume ℓ_a intersects $\ell_b, \ell_c, \ell_{c+1}$ at B, C, E respectively, and $\ell_{a'}$ intersects them at B', C', E' respectively, and ℓ_a intersects $\ell_{a'}$ at I. Since $e_a = \mathsf{a}_{b,c}$, we have

$\mathsf{Area}(\triangle e_a e_b e_c) < \mathsf{Area}(\triangle e_{a'} e_b e_c)$, i.e. $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IBB')$. So, $\mathsf{Area}(\triangle IEE') < \mathsf{Area}(\triangle IBB')$, i.e. $\mathsf{Area}(\triangle e_a e_b e_{c+1}) < \mathsf{Area}(\triangle e_{a'} e_b e_{c+1})$. This means $\mathsf{a}_{b,c+1} \neq e_{a'}$.

2. When $\mathsf{D}_b = \mathsf{D}_c$, vertex D_{b+1} must also equal D_c and hence $\mathsf{a}_{b,c}, \mathsf{a}_{b+1,c}$ are the same edge (i.e. the previous edge of D_b). Next, assume $\mathsf{D}_b \neq \mathsf{D}_c$. See Figure 14 (b). Let $e_a = \mathsf{a}_{b,c}$. We argue that $\mathsf{a}_{b+1,c} \neq e_{a'}$ for any $e_{a'}$ in $(\mathsf{D}_b \circlearrowleft v_a)$. This clearly implies Claim 2. Let the notation be the same as in the proof of Claim 1. Moreover, assume $\ell_a, \ell_{a'}$ intersect ℓ_{b+1} at F, F' respectively. Similarly, we have $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IBB')$, and so $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IFF')$, i.e. $\mathsf{Area}(\triangle e_a e_{b+1} e_c) < \mathsf{Area}(\triangle e_{a'} e_{b+1} e_c)$. This means $\mathsf{a}_{b+1,c} \neq e_{a'}$. ◀

A generalized definition for back-stable and forw-stable

Previously in Section 2, we define back-stable and forw-stable edges for and only for the all-flush triangles with finite areas. Here we generalize these definitions so that they are defined for all all-flush triangles. (We do not need the following generalized definition for showing our main result, but an alternative killing criterion in Appendix C needs it.)

Recall the half-planes $\mathsf{p}_i, \mathsf{p}_i^C$ delimited by ℓ_i. Denote $\phi_i^+ = \mathsf{p}_{i-1} \cap \mathsf{p}_i^C$ and $\phi_i^- = \mathsf{p}_{i-1}^C \cap \mathsf{p}_i$.

▶ **Definition 21.** Given an all-flush triangle $\triangle e_a e_b e_c$. See Figure 15. We regard
- edge e_b is *back-stable* in $\triangle e_a e_b e_c$, if $e_a \prec e_b$ and $\mathsf{Area}(\mathsf{p}_a \cap \phi_b^-) \leq \mathsf{Area}(\mathsf{p}_c \cap \phi_b^+)$.
- edge e_b is *forw-stable* in $\triangle e_a e_b e_c$, if $e_b \prec e_c$ and $\mathsf{Area}(\mathsf{p}_c \cap \phi_{b+1}^+) \leq \mathsf{Area}(\mathsf{p}_a \cap \phi_{b+1}^-)$.

Symmetrically, we can define back-stable and forw-stable for the other two edges e_a, e_c.

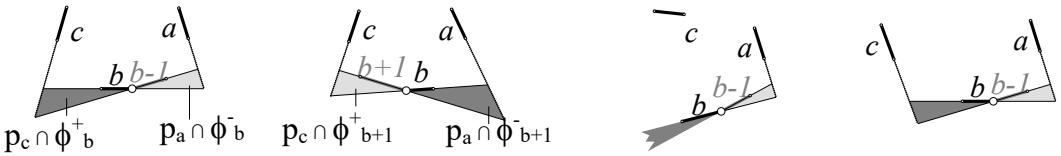

Figure 15 Illustration of the new definition of back-stable and forw-stable. The right two pictures illustrate two cases where $\triangle e_a e_b e_c$ has an infinite area but e_b is back-stable.

Note.
1. As always, we never compare two infinite areas. In the above definition, $\mathsf{Area}(\mathsf{p}_a \cap \phi_b^-)$ is finite when $e_a \prec e_b$; and $\mathsf{Area}(\mathsf{p}_c \cap \phi_{b+1}^+)$ is finite when $e_b \prec e_c$.
2. Edge e_b is back-stable when $b - 1 = a$ – at this time $\mathsf{Area}(\mathsf{p}_a \cap \phi_b^-) = 0$.
   Edge e_b is forw-stable when $b + 1 = c$ – at this time $\mathsf{Area}(\mathsf{p}_c \cap \phi_{b+1}^+) = 0$.
3. This new definition is the same as the previous one for the case $\mathsf{Area}(\triangle e_a e_b e_c) < +\infty$.

▶ **Observation 22** (The extended and full version of Observation 11)**.**
1. *Assume e_b is back-stable in $\triangle e_a e_b e_c$ (perhaps not with a finite area). Then,*
   a. *it is also back-stable in $\triangle e_{a+1} e_b e_c$ when $a + 1 \neq b$.*
   b. *it is also back-stable in $\triangle e_a e_b e_{c+1}$ when $c + 1 \neq a$.*
2. *Assume e_b is forw-stable in $\triangle e_a e_b e_c$ (perhaps not with a finite area). Then,*
   a. *it is also forw-stable in $\triangle e_{a-1} e_b e_c$ when $a - 1 \neq c$.*
   b. *it is also forw-stable in $\triangle e_a e_b e_{c-1}$ when $c - 1 \neq b$.*

Proof. Part 2 is symmetric to Part 1; we only show the proof of Part 1. See Figure 2. Assume e_b is back-stable in $\triangle e_a e_b e_c$. So, $e_a \prec e_b$ and $\mathsf{Area}(\mathsf{p}_a \cap \phi_b^-) \leq \mathsf{Area}(\mathsf{p}_c \cap \phi_b^+)$.

Assume $a + 1 \neq b$. We know $e_{a+1} \prec e_b$ since $e_a \prec e_b$ and $a + 1 \neq b$. Moreover, we know $\mathsf{Area}(\mathsf{p}_{a+1} \cap \phi_b^-) \leq \mathsf{Area}(\mathsf{p}_c \cap \phi_b^+)$ since $\mathsf{Area}(\mathsf{p}_{a+1} \cap \phi_b^-) < \mathsf{Area}(\mathsf{p}_a \cap \phi_b^-)$. Thus claim a holds.

Assume $c + 1 \neq a$. We know $\mathsf{Area}(\mathsf{p}_a \cap \phi_b^-) \leq \mathsf{Area}(\mathsf{p}_{c+1} \cap \phi_b^+)$ since $\mathsf{Area}(\mathsf{p}_c \cap \phi_b^+) < \mathsf{Area}(\mathsf{p}_{c+1} \cap \phi_b^+)$ (or both of these areas are infinite). Further since $e_a \prec e_b$, claim b holds. ◀

B Some supplements

This appendix provides some information to justify some claims in the main body.

```
1  (r, s, t) ← (1, 2, 3);
2  foreach b = 2 to n do
3  |  compute c = a_{r,b};
4  |  if Area(△e_r e_b e_c) < Area(△e_r e_s e_t) then
5  |  |  (s, t) ← (b, c);
6  |  end
7  end
8  // The above is the first step to find one 3-stable triangle; it finds a "2-stable" triangle.
9  while r + 1 ≠ s and Area(△e_{r+1} e_s e_t) < Area(△e_r e_s e_t) do
10 |  r ← r + 1;
11 |  repeat
12 |  |  while s + 1 ≠ t and Area(△e_r e_{s+1} e_t) < Area(△e_r e_s e_t) do s ← s + 1;
13 |  |  while t + 1 ≠ r and Area(△e_r e_s e_{t+1}) < Area(△e_r e_s e_t) do t ← t + 1;
14 |  until none of the above two conditions hold;
15 end
16 // The above is exactly Algorithm 1.
17 while r − 1 ≠ t and Area(△e_{r−1} e_s e_t) < Area(△e_r e_s e_t) do
18 |  r ← r − 1;
19 |  repeat
20 |  |  while s − 1 ≠ r and Area(△e_r e_{s−1} e_t) < Area(△e_r e_s e_t) do s ← s − 1;
21 |  |  while t − 1 ≠ s and Area(△e_r e_s e_{t−1}) < Area(△e_r e_s e_t) do t ← t − 1;
22 |  until none of the above two conditions hold;
23 end
24 // The above while-do sentence is the symmetric subroutine of Algorithm 1.
25 // So far, we have found one 3-stable triangle △e_r e_s e_t.

26 // The following is the Rotate-and-Kill process. All the 3-stable triangles will be
     outputted by this process. (But it may report some other triangles.)
27 (b, c) ← (s, t);
28 repeat
29 |  Compute e_a = a_{b,c};
30 |  if △e_a e_b e_c is 3-stable then
31 |  |  Output △e_a e_b e_c;
32 |  end
33 |  // Unnecessary to check 3-stable if we only care about the optimum triangle.
34 |  if Kill(b, c) = b then
35 |  |  b ← b + 1;
36 |  else
37 |  |  c ← c + 1;
38 |  end
39 |  // The function Kill(b, c) is implemented in Algorithm 2.
40 until (b, c) = (t, r);
```

Algorithm 3: Find all 3-stable triangles.

Note: We have to compute $\mathsf{a}_{b,c}$ incrementally (using Lemma 9 and Lemma 10).

♠ **A reduction from MFT to MAT which is wrong.** It is claimed in [3, 18] that computing the MFT is the dual problem of computing the MAT. We believe that this is only from the combinatorial perspective. There is no evidence that an instance of the MFT problem can be reduced (whether in linear time or not) to an instance of the MAT problem.

In the following we point out an intuitive but wrong reduction from MFT to MAT.

Assume the n edges of P are $e_1, \ldots, e_n$. Let P^* denote the dual polygon of P at some point O, whose vertices are $V_1^*, \ldots, V_n^*$. To compute the MFT circumscribing P, we first compute the MAT in P^*, and then answer $\triangle e_i e_j e_k$ provide that the MAT in P^* is $\triangle V_i^* V_j^* V_k^*$.

A counterexample is given in Figure 16, where the MAT is $V_1^* V_2^* V_4^*$ and MFT is $e_1 e_3 e_4$.

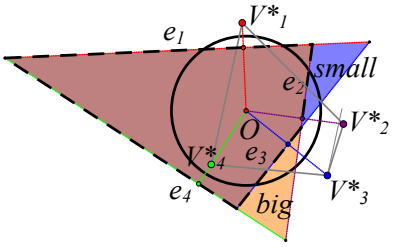

Figure 16 The above reduction is wrong.

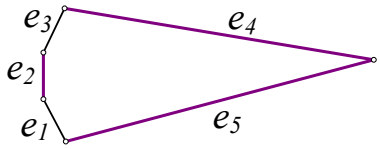

Figure 17 Finiteness condition is necessary when defining 3-stable.

♠ **Why we need finiteness in defining 3-stable triangles.** Recall Definition 4. See Figure 17. If the finiteness condition is removed, $\triangle e_1 e_2 e_3$ would be 3-stable in this example, yet it does not interleave the other 3-stable triangle $\triangle e_2 e_4 e_5$.

♠ **An illustration: from the key inequality to the monotonicity of d.** The key inequality $d(L^{GH}_{b',c'}) \geq d(L^{HG}_{b,c})$ stated in the proof of Lemma 20 implies the monotonicity of d claimed in that lemma. This is illustrated by the examples in Figure 18.

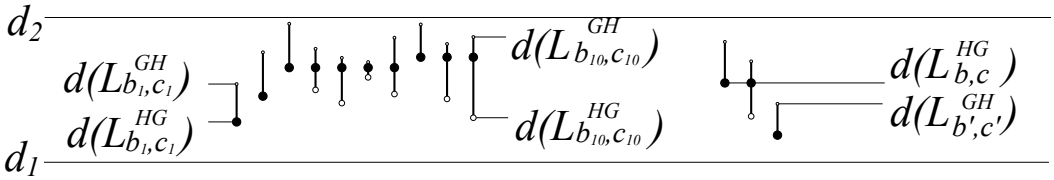

Figure 18 The left picture shows how d is increased in each iteration according to (3). The right picture shows that we cannot get the monotonicity of d if $d(L^{GH}_{b',c'}) \geq d(L^{HG}_{b,c})$ was not true.

♠ **An omitted proof: Case 4 ($b' = b, c' \geq c+2$) in the proof of $d(L^{GH}_{b',c'}) \geq d(L^{HG}_{b,c})$.** Case 4 is symmetric to Case 3. For completeness, we present its proof in this appendix.

Proof. See Figure 19 (a). Let A^+, B^+ be the reflection of A, B around M^+ respectively. Let L be the tangent line of $G^-_{b,c}$ that passes through A^+, and L' the tangent line of $H^-_{b',c'}$ that passes through B^+. As in the previous cases, it reduces to showing that $d(L') > d(L)$.

See Figure 19 (b). Make a parallel line L'_2 of L' at A^+, which intersects ℓ_b, ℓ_{b+1} at A_1, A_2 respectively. It reduces to showing that the area bounded by L'_2, l_b, l_{b+1}, namely $\mathsf{Area}(\triangle v_{b+1} A_1 A_2)$, is smaller than the area bounded by L, l_b, l_{b+1}. The latter equals to $\mathsf{Area}(G^-_{b,c}) = \mathsf{Area}(\triangle v_{b+1} A X)$, where $X = \ell_{c+1} \cap \ell_b$. Let X^+ be the reflection of X around

M^+. Assume the parallel line of L' at X^+ intersects ℓ_b, ℓ_{b+1} at X_1, X_2 respectively. Let $D = \ell_{c'} \cap \ell_{b+1}$, and E be the point on ℓ_{b+1} so that $\overline{XE}$ is parallel to $\overline{BD}$. Clearly, $\mathsf{Area}(\triangle v_{b+1}A_1A_2) < \mathsf{Area}(\triangle v_{b+1}X_1X_2)$ and $\mathsf{Area}(\triangle v_{b+1}AX) > \mathsf{Area}(\triangle v_{b+1}EX)$. So it further reduces to proving that (I) $\mathsf{Area}(\triangle v_{b+1}X_1X_2) < \mathsf{Area}(\triangle v_{b+1}EX)$.

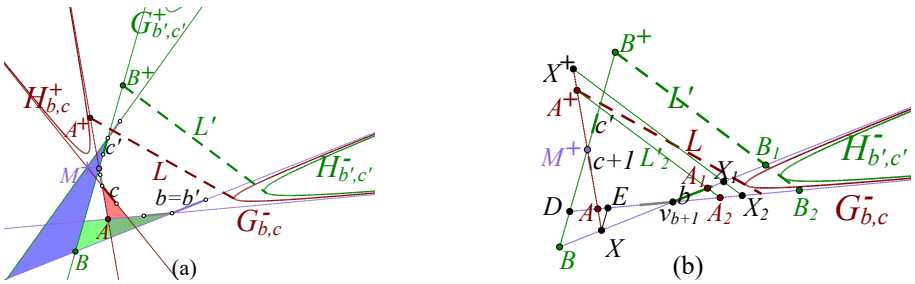

Figure 19 Proof of Lemma 20 - part IV.

Assume L' intersects ℓ_b, ℓ_{b+1} at B_1, B_2. We know (a) $\mathsf{Area}(\triangle v_{b+1}B_1B_2) = \mathsf{Area}(\triangle v_{b+1}DB)$ since L' is tangent to $H^-_{b',c'}$, and so (b): $|v_{b+1}B_1| < |v_{b+1}B|$. Since segment $\overline{B^+X^+}$ is a translate of $\overline{XB}$, (c) $|BX| = |B_1X_1|$. Combining (b) with (c), $|B_1X_1|/|v_{b+1}B_1| > |BX|/|v_{b+1}B|$, so (d) $|v_{b+1}X_1|/|v_{b+1}B_1| < |v_{b+1}X|/|v_{b+1}B|$. Further since $\overline{X_1X_2}$ is parallel to $\overline{B_1B_2}$ whereas $\overline{XE}$ is parallel to $\overline{BD}$, fact (a) and (d) together imply (I). ◀

♠ **Direction $d(L^*_{b,c})$ is not monotone with respect to b or c.** Recall the analysis below Lemma 18. We may guess that $d(L^{GG}_{b,c})$, $d(L^{HH}_{b,c})$, $d(L^{HG}_{b,c})$ and $d(L^{GH}_{b,c})$ increase monotonously when both of b, c keep increasing. This is **not** true; see counterexamples in Figure 20.

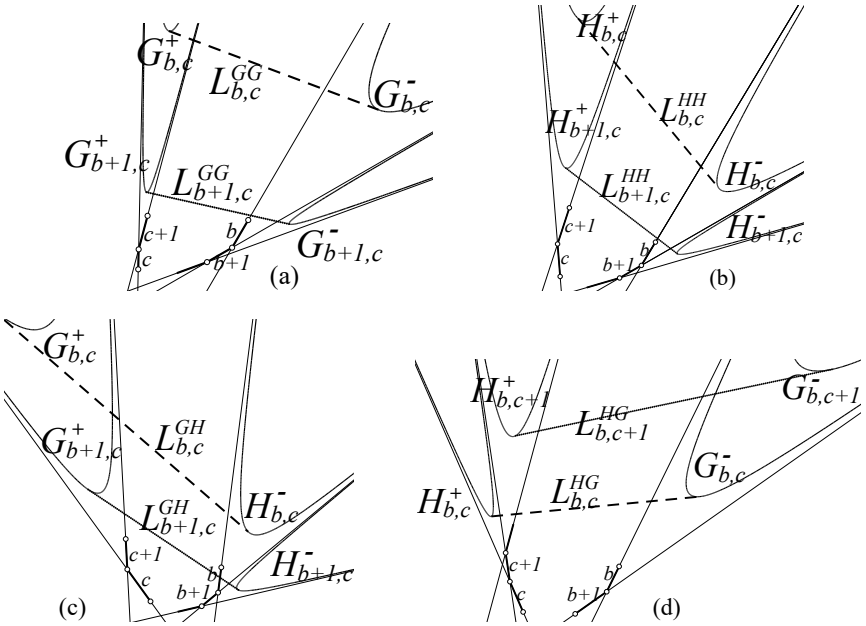

Figure 20 In picture (a), $d(L^{GG}_{b+1,c}) < d(L^{GG}_{b,c})$. In picture (b), $d(L^{HH}_{b+1,c}) < d(L^{HH}_{b,c})$. In picture (c), $d(L^{GH}_{b+1,c}) < d(L^{GH}_{b,c})$. In picture (d), $d(L^{HG}_{b,c+1}) < d(L^{HG}_{b,c})$.

C An alternative $O(\log n)$ time criterion

This appendix presents another killing criterion. It costs $O(\log n)$ time and is less interesting than the $O(1)$ one, but is simpler, and it is the first nontrivial criterion we have discovered. This criterion uses two simple sufficient conditions of (1) and (2) given in Lemma 26.

In this appendix, **assume (b, c) are given as in Lemma 17**. Let $\leq_c$ denote the order of edges in list $\{e_c, \ldots, e_{c-1}\}$ (enumerated clockwise). Denote $\leq_c$ by $\leq$ for short.

Recall the notions back-stable and forw-stable in Appendix A; see Figure 15.

▶ **Definition 23.** Assume $e_j \prec e_k$. Consider $e_{j+1}, \ldots, e_{k-1}$ in clockwise order. Denote
- $\mathsf{x}_{j,k}$ as the first e_i so that e_j is back-stable in $\triangle e_i e_j e_k$ (such edge exists, e.g. e_{j-1}).
- $\mathsf{x}'_{j,k}$ as the last e_i so that e_j is forw-stable in $\triangle e_i e_j e_k$ if any, otherwise e_k.
- $\mathsf{y}_{j,k}$ as the first e_i so that e_k is back-stable in $\triangle e_i e_j e_k$ if any, otherwise e_j.
- $\mathsf{y}'_{j,k}$ as the last e_i so that e_k is forw-stable in $\triangle e_i e_j e_k$ (such edge exists, e.g. e_{k+1}).

▶ **Observation 24.** *Assume $(e_j, e_k) \in \{(e_b, e_{c+1}), \ldots, (e_b, e_r)\} \cup \{(e_{b+1}, e_c), \ldots, (e_t, e_c)\}$ and $e_j \prec e_k$. Note that for these (j, k), list $e_{k+1}, \ldots, e_{j-1}$ is a sublist of $e_c, \ldots, e_{c-1}$. Denote*

$$Q_{j,k} = \{e_i \mid e_k \prec e_i, e_i \prec e_j \text{ and } e_j, e_k \text{ are both stable in } \triangle e_i e_j e_k\}. \tag{4}$$

We claim the following inequalities for edge e_i in $Q_{j,k}$.

$$\mathsf{x}_{j,k} \leq e_i \leq \mathsf{x}'_{j,k} \quad \text{and} \quad \mathsf{y}_{j,k} \leq e_i \leq \mathsf{y}'_{j,k}. \tag{5}$$

Proof. Because $Q_{j,k}$ contains e_i, according to (4), e_j is back-stable in $\triangle e_i e_j e_k$. However, by Definition 23, $\mathsf{x}_{j,k}$ is the first edge e_{i^*} in $e_{k+1}, \ldots, e_{j-1}$ such that e_j is back-stable in $e_{i^*} e_j e_k$. This means $\mathsf{x}_{j,k}$ is the smallest edge e_{i^*} such that e_j is back-stable in $\triangle e_{i^*} e_j e_k$. (When we discuss the order of edges, we refer to $\leq_c$ by default.) Therefore $e_i \geq e_{i*} = \mathsf{x}_{j,k}$.

Similarly, we can get the other three inequalities in (5). ◀

▶ **Observation 25.** *1.*

$$\mathsf{x}'_{b,c+1} < \mathsf{x}_{b+1,c} \text{ and } \mathsf{y}'_{b+1,c} < \mathsf{y}_{b,c+1}.$$

2. Let e_{c^*} be the last edge in $e_{c+1}, \ldots, e_r$ so that $e_b \prec e_{c^*}$. Then

$$\mathsf{x}'_{b,c^*} \leq \ldots \leq \mathsf{x}'_{b,c+1} \text{ and } \mathsf{y}_{b,c+1} \leq \ldots \leq \mathsf{y}_{b,c^*}.$$

3. Let e_{b^*} be the last edge in $e_{b+1}, \ldots, e_t$ so that $e_{b^*} \prec e_c$. Then

$$\mathsf{y}'_{b^*,c} \leq \ldots \leq \mathsf{y}'_{b+1,c} \text{ and } \mathsf{x}_{b+1,c} \leq \ldots \leq \mathsf{x}_{b^*,c}.$$

These inequalities are cornerstones of our $O(\log n)$ time killing criterion. We defer their proofs for a moment and present the main idea of our criterion first. Recall (1) and (2).

▶ **Lemma 26.** *1. At least one is true: (a) $\mathsf{x}'_{b,c+1} < \mathsf{y}_{b,c+1}$; or (b) $\mathsf{y}'_{b+1,c} < \mathsf{x}_{b+1,c}$.*
2. (a) implies (1) – which states that $(e_b, e_{c+1}), (e_b, e_{c+2}), \ldots, (e_b, e_r)$ are all dead.
3. (b) implies (2) – which states that $(e_{b+1}, e_c), (e_{b+2}, e_c), \ldots (e_t, e_c)$ are all dead.

Proof. 1. Assume (a) fails, we argue that (b) holds. According to Observation 25.1, $\mathsf{y}'_{b+1,c} < \mathsf{y}_{b,c+1}$ and $\mathsf{x}'_{b,c+1} < \mathsf{x}_{b+1,c}$. Since (a) fails, $\mathsf{y}_{b,c+1} \leq \mathsf{x}'_{b,c+1}$. Altogether, $\mathsf{y}'_{b+1,c} < \mathsf{x}_{b+1,c}$.

2. Assume (a) holds, i.e. $\mathsf{x}'_{b,c+1} < \mathsf{y}_{b,c+1}$. Applying Observation 25.2, $\mathsf{x}'_{b,c+2} < \mathsf{y}_{b,c+2}$, $\ldots$, $\mathsf{x}'_{b,c^*} < \mathsf{y}_{b,c^*}$. See the top picture in Figure 21. Then, applying Observation 24,

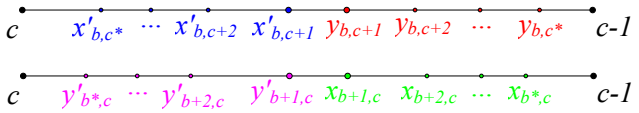

Figure 21 Illustration of the proof of Lemma 26

$Q_{b,c+1}, \ldots, Q_{b,c^*}$ are all empty, hence $(e_b, e_{c+1}), \ldots, (e_b, e_{c^*})$ are dead. Pair $(e_b, e_k) \in \{(e_b, e_{c^*+1}), \ldots, (e_b, e_r)\}$ is dead because e_b is not chasing e_k. Together, (1) holds.

3. Assume (b) holds, i.e. $\mathsf{y}'_{b+1,c} < \mathsf{x}_{b+1,c}$. Applying Observation 25.3, $\mathsf{y}'_{b+2,c} < \mathsf{x}_{b+2,c}$, $\ldots$, $\mathsf{y}'_{b^*,c} < \mathsf{x}_{b^*,c}$. See the bottom picture in Figure 21. Then, applying Observation 24, $Q_{b+1,c}, \ldots, Q_{b^*,c}$ are all empty, hence $(e_{b+1}, e_c), \ldots, (e_{b^*}, e_c)$ are dead. Pair $(e_j, e_c) \in \{(e_{b^*+1}, e_c), \ldots, (e_t, e_c)\}$ is dead because e_j is not chasing e_c. Together, (2) holds. ◀

An $O(\log n)$ time killing criterion.

For trivial cases where (b, c) dissatisfies the assumption in Lemma 17, we use the same method given in Section 3. Otherwise, we compute condition (a) in Lemma 26. If (a) occurs, (1) holds by Lemma 26.2, thus we can safely kill b. Otherwise (b) must hold according to Lemma 26.1, so (2) holds by Lemma 26.3, thus we can safely kill c. (Symmetrically, we can use (b) to be the criterion.) Note that (a) (or (b)) can easily be computed in $O(\log n)$ time - each edge in Definition 23 can be computed by a binary search using Observation 22.

▶ Remark. We have tried but failed to improve the the above criterion to amortized $O(1)$ time. Though $\mathsf{x}, \mathsf{x}', \mathsf{y}, \mathsf{y}'$ has some good monotonicities, they do not monotonously increase when b, c increases. Thus it is not easy to compute them in amortized $O(1)$ time.

Proof of Observation 25.

1. (a) $\mathsf{x}_{b+1,c} > \mathsf{x}'_{b,c+1}$.

First, we claim (i) $\mathsf{x}_{b+1,c} \geq e_{c+2}$. Because $e_{b+1} \prec e_{c+1}$, e_{b+1} cannot be back-stable in $\triangle e_{c+1} e_{b+1} e_c$. So $\mathsf{x}_{b+1,c} \neq e_{c+1}$. Further since $\mathsf{x}_{b+1,c} \in \{e_{c+1}, \ldots, e_b\}$, (i) holds.

By the definition of $\mathsf{x}'_{b,c+1}$, it either equals the largest edge e_a so that e_b is forw-stable in $\triangle e_a e_b e_{c+1}$; or equals e_{c+1} if no such edge exists. Under the second case, inequality (a) holds according to (i). Under the first case, we argue that $\mathsf{x}_{b+1,c} > e_a$. Let I, B, B', C, C', E, E' be the intersections as shown in Figure 22 (a). Because e_b is forw-stable in $\triangle e_a e_b e_{c+1}$, we have $\mathsf{Area}(\triangle ICC') \leq \mathsf{Area}(\triangle IBB')$. Therefore, $\mathsf{Area}(\triangle IEE') < \mathsf{Area}(\triangle IBB')$. Now consider the other pair (e_{b+1}, e_c) and the edge $\mathsf{x}_{b+1,c}$. Because $\mathsf{Area}(\triangle IBB') > \mathsf{Area}(\triangle IEE')$, any edge e_i so that e_{b+1} is back-stable in $\triangle e_i e_{b+1} e_c$, including $\mathsf{x}_{b+1,c}$, must be larger than e_a.

(b) $\mathsf{y}_{b,c+1} > \mathsf{y}'_{b+1,c}$.

By the definition of $\mathsf{y}'_{b+1,c}$, it is the largest edge e_a so that e_c is forw-stable in $\triangle e_a e_{b+1} e_c$. If $e_a \leq e_{c+1}$, inequality (b) holds because $\mathsf{y}_{b,c+1} \geq e_{c+2}$. Assume now $e_a \geq e_{c+2}$, we shall prove $\mathsf{y}_{b,c+1} > e_a$. Let I, B, B', C, C', F, F' be the intersections as shown in Figure 22 (b). Because e_c is forw-stable in $\triangle e_a e_{b+1} e_c$ and $e_a \geq e_{c+2}$, we have $\mathsf{Area}(\triangle ICC') \leq \mathsf{Area}(\triangle IBB')$. So $\mathsf{Area}(\triangle ICC') < +\infty$ and so $e_c \prec e_a$. Further since $e_b \prec e_c$, we get $e_a \neq e_b$, so $e_b > e_a$. Also because $\mathsf{Area}(\triangle ICC') \leq \mathsf{Area}(\triangle IBB')$, we have $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IFF')$.

Now consider the other pair (e_b, e_{c+1}) and edge $\mathsf{y}_{b,c+1}$. By definition, $\mathsf{y}_{b,c+1}$ is either the smallest e_i so that e_{c+1} is back-stable in $\triangle e_i e_b e_{c+1}$, or equals e_b. Under the second case, $\mathsf{y}_{b,c+1} = e_b > e_a$. Under the first case, because $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IFF')$, any edge e_i so that e_{c+1} is back-stable in $\triangle e_i e_b e_{c+1}$, including $\mathsf{y}_{b,c+1}$, must be larger than e_a.

2. **Assume now e_{k-1}, e_k are two consecutive elements in list $e_{c+1}, \ldots, e_{c^*}$.**

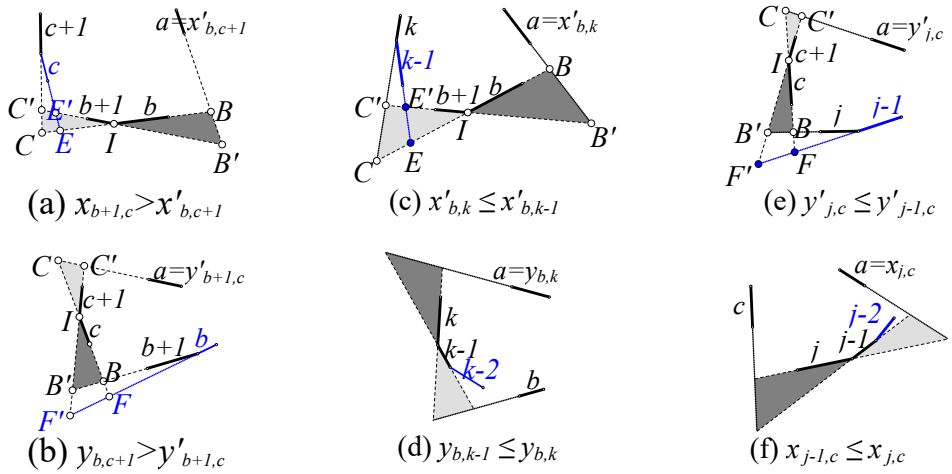

Figure 22 Illustration of the proof of Observation 25

Note that e_b is chasing all edges in this list, so $e_b \prec e_{k-1}$ and $e_b \prec e_k$.

(c) $\mathsf{x}'_{b,k} \leq \mathsf{x}'_{b,k-1}$.

First, we claim (i) $\mathsf{x}'_{b,k-1} \geq e_k$. Because $e_b \prec e_{k-1}$ and $e_b \prec e_k$, we know $e_{b+1} \prec e_k$. This means edge e_b is forw-stable in $\triangle e_k e_b e_{k-1}$, which implies (i).

By the definition of $\mathsf{x}'_{b,k}$, it either equals the largest edge e_a so that e_b is forw-stable in $\triangle e_a e_b e_k$, or equals e_k. Under the second case, $\mathsf{x}'_{b,k} = e_k \leq \mathsf{x}'_{b,k-1}$ according to (i). Under the first case, we shall prove that $e_a \leq \mathsf{x}'_{b,k-1}$. Let I, B, B', C, C', E, E' be intersections as shown in Figure 22 (c). Because e_b is forw-stable in $\triangle e_a e_b e_k$, $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IBB')$. Therefore $\mathsf{Area}(\triangle IEE') < \mathsf{Area}(\triangle IBB')$. Now consider the other pair (e_b, e_{k-1}) and edge $\mathsf{x}'_{b,k-1}$. Because $\mathsf{Area}(\triangle IEE') < \mathsf{Area}(\triangle IBB')$, e_b is forw-stable in $\triangle e_a e_b e_{k-1}$. However, $\mathsf{x}'_{b,k-1}$ is the largest e_i so that e_b is forw-stable in $\triangle e_i e_b e_{k-1}$. Therefore $\mathsf{x}'_{b,k-1} \geq e_a$.

(d) $\mathsf{y}_{b,k-1} \leq \mathsf{y}_{b,k}$.

See Figure 22 (d). By the definition of $\mathsf{y}_{b,k}$, it either equals the smallest edge e_a so that e_k is back-stable in $\triangle e_a e_b e_k$, or equals e_b. Under the second case, $\mathsf{y}_{b,k-1} \leq e_b = \mathsf{y}_{b,k}$. Under the first case, applying Lemma 9, e_{k-1} is back-stable in $\triangle e_a e_b e_{k-1}$, and so $\mathsf{y}_{b,k-1} \leq e_a = \mathsf{y}_{b,k}$.

3. **Assume now e_{j-1}, e_j are two consecutive edges in list $e_{b+1}, \ldots, e_{b^*}$.**

Note that all edges in this list are chasing e_{c+1}, so $e_{j-1} \prec e_{c+1}$ and $e_j \prec e_{c+1}$.

(e) $\mathsf{y}'_{j,c} \leq \mathsf{y}'_{j-1,c}$.

By the definition of $\mathsf{y}'_{j,c}$, it equals the largest e_a so that e_c is forw-stable in $\triangle e_a e_j e_c$. Let I, B, B', C, C', F, F' be the intersections as shown in Figure 22 (e). Because e_c is forw-stable in $\triangle e_a e_j e_c$, we get $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IBB')$. So $\mathsf{Area}(\triangle ICC') < +\infty$ and hence $e_a \prec e_c$. However, $e_{j-1} \prec e_c$ because $e_{j-1} \prec e_{c+1}$ and $e_j \prec e_{c+1}$. Therefore, $a \neq j-1$. Also because $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IBB')$, we have $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IFF')$.

Now consider edge pair (e_{j-1}, e_c) and edge $\mathsf{y}'_{j-1,c}$. Since $\mathsf{Area}(\triangle ICC') < \mathsf{Area}(\triangle IFF')$, edge e_c is forw-stable in $\triangle e_a e_{j-1} e_c$. Therefore, the largest edge e_i so that e_c is forw-stable in $\triangle e_i e_{j-1} e_c$, which is defined as $\mathsf{y}'_{j-1,c}$, must be larger than e_a.

(f) $\mathsf{x}_{j-1,c} \leq \mathsf{x}_{j,c}$.

See Figure 22 (f). By the definition of $\mathsf{x}_{j,c}$, it equals the smallest edge e_a so that e_j is back-stable in $\triangle e_a e_j e_c$. If $e_a = e_{j-1}$, we know $\mathsf{x}_{j-1,c} < e_{j-1} = e_a$. Otherwise, since e_j is back-stable, applying Lemma 9, e_{j-1} is back-stable in $\triangle e_a e_{j-1} e_c$ and so $\mathsf{x}_{j-1,c} \leq e_a$. ◀

References

1. A. Aggarwal, J.S. Chang, and C.K. Yap. Minimum area circumscribing polygons. *The Visual Computer*, 1(2):112–117, Aug 1985. doi:10.1007/BF01898354.
2. A. Aggarwal, M. M. Klawe, S. Moran, P. Shor, and R. Wilber. Geometric applications of a matrix-searching algorithm. *Algorithmica*, 2(1-4):195–208, 1987.
3. A. Aggarwal, B. Schieber, and T. Tokuyama. Finding a minimum-weight k-link path in graphs with the concave monge property and applications. *Discrete & Computational Geometry*, 12(1):263–280, 1994.
4. B. Bhattacharya and A. Mukhopadhyay. *On the Minimum Perimeter Triangle Enclosing a Convex Polygon*, pages 84–96. Springer Berlin Heidelberg, 2003. doi:10.1007/978-3-540-44400-8_9.
5. J. E. Boyce, D. P. Dobkin, R. L. (Scot) Drysdale, III, and L. J. Guibas. Finding extremal polygons. In *14th Symposium on Theory of Computing*, pages 282–289, 1982.
6. P. Brass and H. Na. Finding the maximum bounded intersection of k out of n halfplanes. *Information Processing Letters*, 110(3):113 – 115, 2010.
7. S. Chandran and D. M. Mount. A paraellel algorithm for enclosed and enclosing triangles. *International Journal of Computational Geometry & Applications*, 02(02):191–214, 1992. doi:10.1142/S0218195992000123.
8. J. S. Chang and C. K. Yap. A polynomial solution for potato-peeling and other polygon inclusion and enclosure problems. In *25th Annual Symposium on Foundations of Computer Science*, pages 408–416, Oct 1984. doi:10.1109/SFCS.1984.715942.
9. D. P. Dobkin and L. Snyder. On a general method for maximizing and minimizing among certain geometric problems. In *20th Annual Symposium on Foundations of Computer Science*, pages 9–17, Oct 1979. doi:10.1109/SFCS.1979.28.
10. K. Jin. Maximal parallelograms in convex polygons - a novel geometric structure. *CoRR*, abs/1512.03897, 2015. URL: http://arxiv.org/abs/1512.03897.
11. K. Jin. Maximal area triangles in a convex polygon. *CoRR*, abs/1707.04071, 2017. URL: http://arxiv.org/abs/1707.04071.
12. Y. Kallus. A linear-time algorithm for the maximum-area inscribed triangle in a convex polygon. *CoRR*, abs/1706.03049, 2017.
13. V. Keikha, M. Löffler, J. Urhausen, and I. v. d. Hoog. Maximum-area triangle in a convex polygon, revisited. *CoRR*, abs/1705.11035, 2017.
14. V. Klee and M. C. Laskowski. Finding the smallest triangles containing a given convex polygon. *Journal of Algorithms*, 6(3):359 – 375, 1985. doi:https://doi.org/10.1016/0196-6774(85)90005-7.
15. J. S.B. Mitchell and V. Polishchuk. Minimum-perimeter enclosures. *Information Processing Letters*, 107(3):120 – 124, 2008. doi:https://doi.org/10.1016/j.ipl.2008.02.007.
16. J. O'Rourke, A. Aggarwal, S. Maddila, and M. Baldwin. An optimal algorithm for finding minimal enclosing triangles. *Journal of Algorithms*, 7(2):258 – 269, 1986. doi:http://dx.doi.org/10.1016/0196-6774(86)90007-6.
17. N. A. D. Pano, Y. Ke, and J. O'Rourke. Finding largest inscribed equilateral triangles and squares. In *Proceeding of Annual Allerton Conference on Communication, Control, and Computing*, pages 869–878, 1987.
18. B. Schieber. Computing a minimum-weight k-link path in graphs with the concave monge property. In *Proceedings of the Sixth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '95, pages 405–411. Society for Industrial and Applied Mathematics, 1995.
19. G. Toussaint. Solving geometric problems with the rotating calipers. In *In Proc. IEEE MELECON'83*, pages 10–02, 1983.
20. F. Vivien and N. Wicker. Minimal enclosing parallelepiped in 3d. *Computational Geometry*, 29(3):177 – 190, 2004. doi:https://doi.org/10.1016/j.comgeo.2004.01.009.

21 Y. Zhou and S. Suri. Algorithms for a minimum volume enclosing simplex in three dimensions. *SIAM Journal on Computing*, 31(5):1339–1357, 2002. doi:10.1137/S0097539799363992.